

ระบบแนะนำคอร์สเรียน
คอร์สดีบอกต่อ
COURSE RECOMMENDATION SYSTEM
COURSEHUB

นายศุภวัฒน์ ข่ายทอง	รหัสนักศึกษา 673380062-2
นายสรวิชญ์ วันเสน	รหัสนักศึกษา 673380064-8
นายเกียรติศักดิ์ นันทรัตน์	รหัสนักศึกษา 673380072-9
นายภาคิน เมฆสุวรรณ	รหัสนักศึกษา 673380054-1

เสนอ

รศ.ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353002

Principles of Software Design and Development

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and Development สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น จัดทำขึ้นเพื่อนำเสนอการวิเคราะห์ การออกแบบ การพัฒนา และการทดสอบระบบแนะนำคอร์สเรียน คอร์สดีบอกต่อ หรือ CourseHub ซึ่งเป็นเว็บแอปพลิเคชันตัวกลางที่รวบรวมคอร์สเรียนออนไลน์จากหลายแพลตฟอร์มไว้ในที่เดียว ให้ผู้เรียนค้นหา เปรียบเทียบ บันทึก รีวิว และรับคำแนะนำคอร์สที่ตรงกับงบประมาณ เวลา และระดับความรู้ของตนเอง

เนื้อหาของรายงานแบ่งออกเป็น 5 บท ได้แก่ บทนำ ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง การวิเคราะห์และออกแบบระบบ ผลการพัฒนาและการทดสอบระบบ และสรุปผล อภิปรายผล และข้อเสนอแนะ โดยอธิบายการนำสถาปัตยกรรมแบบหลายชั้น หลักการ SOLID และรูปแบบการออกแบบกลุ่ม Behavioral ได้แก่ Strategy State และ Observer มาใช้แก้ปัญหาจริงของระบบ พร้อมผลการทดสอบและการนำระบบขึ้นใช้งานบนคลาวด์

คณะผู้จัดทำขอขอบพระคุณ รศ.ดร.ปัญญาพล หอระตะ อาจารย์ผู้สอนรายวิชา ที่ให้ความรู้และคำแนะนำตลอดภาคเรียน และหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจการออกแบบและพัฒนาซอฟต์แวร์ หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยไว้ ณ ที่นี้

คณะผู้จัดทำ

10 ตุลาคม 2569

สารบัญ

หน้า

คำนำ.....ก
สารบัญตาราง.....ง
สารบัญภาพ.....จ
บทที่ 1 บทนำ.....1
1.1 ความเป็นมาและความสำคัญของปัญหา.....1
1.2 วัตถุประสงค์ของโครงงาน.....1
1.3 ขอบเขตของโครงงาน.....2
1.4 ประโยชน์ที่คาดว่าจะได้รับ.....2
1.5 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา.....3
1.6 สมาชิกและหน้าที่รับผิดชอบ.....3
1.7 แผนการดำเนินงาน.....4
1.8 นิยามศัพท์เฉพาะ.....5
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง.....6
2.1 สถาปัตยกรรมแบบหลายชั้น.....6
2.2 หลักการ SOLID.....6
2.3 รูปแบบการออกแบบของ GoF.....7
2.4 REST API และรหัสสถานะ HTTP.....7
2.5 ฐานข้อมูลเชิงสัมพันธ์ ORM และ Database Migration.....8
2.6 ความปลอดภัยของเว็บแอปพลิเคชัน.....9
2.7 Single Page Application และ React.....9
2.8 การทดสอบซอฟต์แวร์.....9
2.9 Container, CI/CD และบริการคลาวด์.....10
บทที่ 3 การวิเคราะห์และออกแบบระบบ.....11
3.1 การวิเคราะห์ความต้องการของระบบ.....11
3.2 การออกแบบสถาปัตยกรรมระบบ.....13
3.3 การออกแบบฐานข้อมูล.....15
3.4 การออกแบบ API.....18
3.5 การประยุกต์ใช้รูปแบบการออกแบบ.....19
3.6 การออกแบบความปลอดภัยและสิทธิ์การเข้าถึง.....24
3.7 การออกแบบกระบวนการ CI/CD.....24
บทที่ 4 ผลการพัฒนาและการทดสอบระบบ.....26
4.1 ผลการพัฒนาระบบ.....26

4.2 ผลการทดสอบระบบ....32
4.3 ผลของกระบวนการ CI/CD....36
4.4 ผลการทำงานร่วมกันผ่าน Git....36
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ....37
5.1 สรุปผลการดำเนินงาน....37
5.2 อภิปรายผล....37
5.3 ปัญหาและอุปสรรค....38
5.4 ข้อเสนอแนะและแนวทางพัฒนาต่อ....38
บรรณานุกรม....39
ภาคผนวก....41
ภาคผนวก ก การติดตั้งและเรียกใช้งานระบบ....42

สารบัญตาราง

ตารางที่ **หน้า**

1.1 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาระบบ....3
1.2 สมาชิกและหน้าที่รับผิดชอบ....4
1.3 แผนการดำเนินงานแบ่งตามระยะ....5
2.1 รหัสสถานะ HTTP ที่ระบบใช้....8
3.1 รายการกรณีการใช้งานและสถานะการพัฒนา....12
3.2 ตารางในฐานข้อมูลและหน้าที่....17
3.3 ไฟล์ migration ของ Flyway....18
3.4 endpoint หลักของระบบ....18
3.5 คำสั่งที่อนุญาตในแต่ละสถานะของคอร์ส....20
3.6 สูตรคะแนนของแต่ละ Strategy....22
3.7 ตัวอย่างการประยุกต์ใช้หลักการ SOLID ในระบบ....24
4.1 ผลการทดสอบอัตโนมัติ....32
4.2 ตัวอย่างเกณฑ์การยอมรับและผลการทดสอบ....33
4.3 การคำนวณคะแนนของคอร์สอันดับที่ 1....34
4.4 สรุปผลการทดสอบการยอมรับบนระบบจริง....35
4.5 จำนวน commit ของสมาชิกแต่ละคน....36

สารบัญภาพ

ภาพที่ **หน้า**

3.1 แผนภาพกรณีการใช้งานของระบบ CourseHub....11
3.2 สถาปัตยกรรมการติดตั้งระบบบนคลาวด์....14
3.3 สถาปัตยกรรมแบบหลายชั้นของ Backend....15
3.4 แผนภาพความสัมพันธ์ของข้อมูลในระบบ CourseHub....16
3.5 แผนภาพสถานะของคอร์ส....20
3.6 แผนภาพคลาสของ Strategy Pattern ในส่วน Course Matcher....21
3.7 แผนภาพคลาสของ Observer Pattern....23
3.8 กระบวนการ CI/CD ด้วย GitHub Actions....25
4.1 หน้าแรกของระบบ CourseHub....26
4.2 หน้าสำรวจและค้นหาคอร์ส....27
4.3 หน้าแบบทดสอบ Course Matcher....28
4.4 หน้าเข้าสู่ระบบ....29
4.5 หน้ารีวิวของคอร์ส....29
4.6 แบบฟอร์มสร้างคอร์สแบบร่าง....30
4.7 หน้าจัดการสมาชิกทีมของผู้ให้บริการ....31
4.8 หน้าแจ้งการเชื่อมต่อไม่สำเร็จพร้อมปุ่มลองใหม่....32

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ปัจจุบันคอร์สเรียนออนไลน์มีจำนวนมากและกระจายอยู่บนหลายแพลตฟอร์ม เช่น Coursera Udemy Chula MOOC FutureSkill และ edX โดยแต่ละแพลตฟอร์มแสดงข้อมูลราคา ระยะเวลาเรียน ภาษา และระดับความยากในรูปแบบที่แตกต่างกัน ผู้เรียนจึงต้องเปิดเว็บไซต์เปรียบเทียบทีละแห่ง ทำให้เสียเวลาและเลือกคอร์สที่ตรงกับงบประมาณ เวลาที่มี และพื้นฐานความรู้ของตนเองได้ยาก อีกทั้งรีวิวที่พบบนอินเทอร์เน็ตจำนวนมากยังไม่ผ่านการคัดกรอง จึงไม่สามารถใช้ประกอบการตัดสินใจได้อย่างมั่นใจ

คณะผู้จัดทำจึงพัฒนาระบบ CourseHub หรือคอร์สดีบอกต่อ ซึ่งเป็นเว็บแอปพลิเคชันตัวกลางที่เชื่อมผู้ใช้ 3 กลุ่ม คือ ผู้เรียน ผู้ให้บริการคอร์ส และผู้ดูแลระบบ ผู้ให้บริการสมัครเปิดบัญชีและลงข้อมูลคอร์สเพื่อส่งให้ผู้ดูแลตรวจก่อนเผยแพร่ ผู้เรียนค้นหา กรอง บันทึกคอร์ส เขียนรีวิว และทำแบบทดสอบเพื่อรับคำแนะนำคอร์สที่เหมาะสมกับตนเอง แล้วกดลิงก์ไปเรียนและชำระเงินที่เว็บไซต์ต้นทาง ส่วนผู้ดูแลระบบทำหน้าที่ตรวจสอบคอร์ส รับรองผู้ให้บริการ คัดกรองรีวิว และตรวจดูประวัติการใช้งานระบบ

นอกจากเป้าหมายด้านการใช้งาน โครงงานนี้ยังเป็นงานตามข้อกำหนดของรายวิชา CP353002 Principles of Software Design and Development ซึ่งกำหนดให้พัฒนาด้วย Spring Boot บนภาษา Java ใช้ฐานข้อมูล SQL ผ่าน Spring Data JPA ออกแบบเป็นสถาปัตยกรรมแบบหลายชั้นที่ไม่เรียกข้ามชั้น ประยุกต์หลักการ SOLID และเลือกใช้รูปแบบการออกแบบของ GoF จากกลุ่มเดียวอย่างน้อย 3 รูปแบบ พร้อมทั้งทดสอบระบบและนำขึ้นใช้งานบนคลาวด์ให้เข้าถึงได้ผ่าน URL สาธารณะ ระบบ CourseHub จึงเป็นทั้งเครื่องมือแก้ปัญหาการเลือกคอร์สเรียนและเป็นกรณีศึกษาของการออกแบบซอฟต์แวร์ตามหลักวิชา

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาเว็บแอปพลิเคชันที่รวบรวมคอร์สเรียนจากหลายแพลตฟอร์ม ให้ผู้เรียนค้นหา กรอง เรียงลำดับ และเปรียบเทียบคอร์สได้ในที่เดียว

1.2.2 เพื่อพัฒนาระบบแนะนำคอร์สแบบกติกาที่จับคู่คอร์สกับหมวดหมู่ ระดับ ภาษา งบประมาณ และเวลาเรียนต่อสัปดาห์ของผู้เรียน พร้อมแสดงเหตุผลประกอบที่ตรวจสอบได้

1.2.3 เพื่อพัฒนากระบวนการตรวจสอบคอร์ส ผู้ให้บริการ และรีวิวโดยผู้ดูแลระบบ ให้ข้อมูลที่เผยแพร่มีความน่าเชื่อถือ

1.2.4 เพื่อประยุกต์ใช้สถาปัตยกรรมแบบหลายชั้น หลักการ SOLID และรูปแบบการออกแบบ Strategy State และ Observer กับปัญหาจริงของระบบ

1.2.5 เพื่อทดสอบระบบหลายระดับ และนำระบบขึ้นใช้งานจริงบนคลาวด์พร้อมกระบวนการ CI/CD อัตโนมัติ

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตด้านผู้ใช้งาน

ระบบรองรับผู้ใช้ 3 กลุ่ม ได้แก่ ผู้เรียนหรือ Learner ซึ่งรวมทั้งผู้เยี่ยมชมที่ยังไม่เข้าสู่ระบบและสมาชิกที่เข้าสู่ระบบแล้ว ผู้ให้บริการคอร์สหรือ Provider ซึ่งแบ่งบทบาทภายในทีมเป็นเจ้าของหรือ Owner และผู้แก้ไขหรือ Editor และผู้ดูแลระบบหรือ Admin

1.3.2 ขอบเขตด้านฟังก์ชัน

ระบบออกแบบกรณีการใช้งานไว้ทั้งหมด 19 กรณี และพัฒนาแล้ว 17 กรณี ครอบคลุมฟังก์ชันหลักดังนี้

1. การสมัครสมาชิก เข้าสู่ระบบ ออกจากระบบ และจัดการโปรไฟล์ส่วนตัว โดยยืนยันตัวตนด้วย session cookie
2. การค้นหา กรอง เรียงลำดับ และแบ่งหน้ารายการคอร์ส โดยแสดงเฉพาะคอร์สที่เผยแพร่แล้วจากผู้ให้บริการที่ได้รับการรับรอง
3. การบันทึกคอร์สที่สนใจ และการเขียนรีวิวที่ให้คะแนนภาพรวม คุณค่าของเนื้อหา คุณภาพการสอน และความตรงตามระดับความยาก
4. แบบทดสอบ Course Matcher ที่แนะนำคอร์สสูงสุด 3 อันดับพร้อมคะแนนและเหตุผล
5. การสมัครเป็นผู้ให้บริการ การจัดการข้อมูลสถาบันและสมาชิกทีม การสร้าง แก้ไข ส่งตรวจ และลบคอร์สแบบร่าง
6. การตรวจอนุมัติคอร์ส การรับรองผู้ให้บริการ การตรวจคัดกรองรีวิว และการดูประวัติการใช้งานระบบของผู้ดูแล

1.3.3 สิ่งที่อยู่นอกขอบเขต

ระบบไม่รับชำระเงินและไม่โฮสต์วิดีโอการสอนเอง ผู้เรียนต้องไปเรียนและชำระเงินที่เว็บไซต์ต้นทาง ระบบแนะนำคอร์สใช้กติกาที่กำหนดไว้ชัดเจนแทนการใช้ปัญญาประดิษฐ์ ส่วนกรณีการใช้งานการดูเส้นทางการเรียนรู้สายอาชีพและการจัดการหมวดหมู่โดยผู้ดูแลยังอยู่ในแผนพัฒนาต่อ ปัจจุบันระบบใช้ข้อมูลหมวดหมู่ตั้งต้นที่บรรจุผ่าน migration

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1.4.1 ผู้เรียนค้นหาและเปรียบเทียบคอร์สจากหลายแพลตฟอร์มได้รวดเร็วขึ้น และได้รับคำแนะนำพร้อมเหตุผลที่ตรวจสอบได้

1.4.2 ผู้ให้บริการมีช่องทางเผยแพร่คอร์สที่ผ่านการตรวจสอบ และจัดการทีมงานตามบทบาทได้

1.4.3 ผู้ดูแลระบบควบคุมคุณภาพของคอร์ส ผู้ให้บริการ และรีวิว และตรวจสอบย้อนหลังได้จากประวัติการใช้งาน

1.4.4 คณะผู้จัดทำได้ฝึกการออกแบบซอฟต์แวร์ด้วยหลักการ SOLID และรูปแบบการออกแบบ การทดสอบหลายระดับ การทำงานร่วมกันผ่าน Git และการนำระบบขึ้นใช้งานบนคลาวด์

1.5 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

คณะผู้จัดทำเลือกเครื่องมือตามข้อกำหนดของรายวิชาและข้อจำกัดด้านงบประมาณ โดยใช้บริการคลาวด์ระดับใช้ฟรีทั้งหมด รายละเอียดแสดงในตารางที่ 1.1

ตารางที่ 1.1 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาระบบ

ส่วนของระบบ	เทคโนโลยี	หน้าที่
Frontend	React 18, TypeScript 5, Vite 6, React Router 7, Tailwind CSS 3	สร้างส่วนติดต่อผู้ใช้แบบ Single Page Application
Backend	Java 21, Spring Boot 3.5, Spring MVC, Spring Security, Spring Data JPA, springdoc OpenAPI	ให้บริการ REST API กฎธุรกิจ และความปลอดภัย
ฐานข้อมูล	PostgreSQL, Flyway, Hibernate	จัดเก็บข้อมูลและควบคุมเวอร์ชันของโครงสร้างฐานข้อมูล
การทดสอบ	JUnit 5, Mockito, MockMvc, Testcontainers, Vitest, React Testing Library, Playwright	ทดสอบระดับหน่วย การทำงานร่วมกัน หน้าจอ และแบบครบวงจร
DevOps	Docker, Docker Compose, GitHub Actions	สร้างสภาพแวดล้อมที่ทำซ้ำได้และกระบวนการ CI/CD
คลาวด์	Vercel Hobby, Render Free, Neon Free	ให้บริการ Frontend Backend และฐานข้อมูลบนอินเทอร์เน็ต
การทำงานร่วมกัน	Git, GitHub, Pull Request	ควบคุมเวอร์ชันและตรวจทานโค้ดก่อนรวมงาน

1.6 สมาชิกและหน้าที่รับผิดชอบ

คณะผู้จัดทำมีสมาชิก 4 คน แต่ละคนพัฒนางานบน branch ของตนเองที่ตั้งชื่อตามรูปแบบชื่อและรหัสนักศึกษา แล้วรวมงานเข้าสู่ branch develop ผ่าน Pull Request ที่มีผู้ตรวจทานอย่างน้อย 1 คน การแบ่งหน้าที่แสดงในตารางที่ 1.2

ตารางที่ 1.2 สมาชิกและหน้าที่รับผิดชอบ

ลำดับ	รหัสนักศึกษา	ชื่อ-สกุล	หน้าที่รับผิดชอบ
1	673380062-2	นายศุภวัฒน์ ข่ายทอง	ระบบสมาชิก การยืนยันตัวตน และโปรไฟล์ทั้งฝั่ง Backend และหน้าเว็บ Matcher Strategy การทดสอบ E2E และหน้าจัดการสมาชิกทีม
2	673380064-8	นายสรวิชญ์ วันเสน	การตรวจอนุมัติคอร์ส การรับรองผู้ให้บริการ การตรวจรีวิว และหน้าประวัติการใช้งานระบบ
3	673380072-9	นายเกียรติศักดิ์ นันทรัตน์	CRUD ผู้ให้บริการและคอร์สทั้งฝั่ง Backend และหน้าเว็บ โครงสร้างฐานข้อมูลด้วย Flyway การสำรองและกู้คืนข้อมูล และการทดสอบการยอมรับบนระบบจริง
4	673380054-1	นายภาคิน เมฆสุวรรณ	หน้าค้นหาคอร์ส หน้าแบบทดสอบและผลแนะนำ การทดสอบการทำงานพร้อมกัน CI/CD การนำระบบขึ้นคลาวด์ และการออกแบบหน้าเว็บ

1.7 แผนการดำเนินงาน

คณะผู้จัดทำเริ่มสร้าง repository เมื่อวันที่ 10 กันยายน 2569 ปรับแผนจากเทคโนโลยีเดิมมาเป็น Spring Boot ตามข้อกำหนดของรายวิชาเมื่อวันที่ 15 กันยายน 2569 และทดสอบการยอมรับบนระบบจริงรอบล่าสุดเมื่อวันที่ 10 ตุลาคม 2569 แผนงานแบ่งเป็น 7 ระยะตามลำดับความสำคัญ ดังตารางที่ 1.3

ตารางที่ 1.3 แผนการดำเนินงานแบ่งตามระยะ

ระยะ	งานหลัก	ผลลัพธ์ที่ใช้ตรวจรับ
P0	กำหนดขอบเขต สมาชิก branch กรณีการใช้งาน และบันทึกการตัดสินใจเชิงสถาปัตยกรรม	README รายชื่อสมาชิก ตารางข้อกำหนด และ ADR
P1	โครงสร้าง repository, Maven, Vite, Docker, schema ด้วย Flyway และพื้นฐานความปลอดภัย	build และ migrate ฐานข้อมูลได้ ระบบสมาชิกเชื่อมกับหน้าเว็บ
P2	CRUD ผู้ให้บริการและคอร์ส สมาชิกทีม การค้นหาคอร์ส และการบันทึกคอร์ส	CRUD 2 resource ทำงานได้ทั้งหน้าเว็บและ API
P3	การตรวจอนุมัติด้วย State รีวิว AuditLog Observer และหน้าผู้ดูแล	workflow ถูกต้อง การแก้ไขชนกันตอบ 409
P4	Matcher ด้วย Strategy แบบทดสอบ การทดสอบ E2E และเอกสารแผนภาพ	สาธิต Strategy State และ Observer ได้
P5	นำระบบขึ้นคลาวด์ Swagger สาธารณะ การสำรองและกู้คืนข้อมูล smoke test และ CI/CD	เข้าใช้งานผ่าน URL สาธารณะได้จริง
P6	ทดสอบการยอมรับ แก้ข้อบกพร่อง จัดทำเอกสาร สไลด์ และซ้อมนำเสนอ	ทุกข้อกำหนดมีหลักฐานประกอบ

1.8 นิยามศัพท์เฉพาะ

1.8.1 ผู้เรียน หมายถึง ผู้ใช้ทั่วไปที่ค้นหาคอร์ส ทำแบบทดสอบ บันทึกคอร์ส และเขียนรีวิว

1.8.2 ผู้ให้บริการ หมายถึง สถาบันหรือผู้สอนที่สมัครเปิดบัญชีเพื่อนำคอร์สมาเผยแพร่บนระบบ

1.8.3 ผู้ดูแลระบบ หมายถึง ผู้ใช้บทบาท ADMIN ที่ตรวจสอบคอร์ส ผู้ให้บริการ และรีวิวก่อนเผยแพร่

1.8.4 คอร์สแบบร่าง หมายถึง คอร์สสถานะ DRAFT ที่ผู้ให้บริการสร้างหรือแก้ไขแล้วแต่ยังไม่ได้ส่งให้ผู้ดูแลตรวจ

1.8.5 Course Matcher หมายถึง ส่วนแนะนำคอร์สที่รับคำตอบจากผู้เรียนแล้วจัดอันดับคอร์สตามกติกาที่กำหนด

1.8.6 Audit Log หมายถึง ประวัติการกระทำสำคัญในระบบ ระบุผู้กระทำ ชนิดข้อมูล สถานะก่อนและหลัง เหตุผล และเวลา

1.8.7 Cold start หมายถึง ช่วงเวลาที่บริการบนคลาวด์ระดับใช้ฟรีต้องเริ่มทำงานใหม่หลังจากหยุดพักเพราะไม่มีผู้ใช้งาน

บทที่ 2
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

บทนี้อธิบายแนวคิดด้านสถาปัตยกรรมซอฟต์แวร์ หลักการออกแบบเชิงวัตถุ รูปแบบการออกแบบ และเทคโนโลยีที่คณะผู้จัดทำนำมาใช้พัฒนาระบบ CourseHub เพื่อเป็นพื้นฐานของการวิเคราะห์และออกแบบในบทที่ 3

2.1 สถาปัตยกรรมแบบหลายชั้น

สถาปัตยกรรมแบบหลายชั้นหรือ Layered Architecture แบ่งซอฟต์แวร์ออกเป็นชั้นตามความรับผิดชอบ แต่ละชั้นเรียกใช้ได้เฉพาะชั้นที่อยู่ถัดลงไป ทำให้การเปลี่ยนแปลงในชั้นหนึ่งกระทบชั้นอื่นน้อยที่สุด Fowler (2002) อธิบายว่าการแยกชั้นการนำเสนอ ชั้นตรรกะทางธุรกิจ และชั้นแหล่งข้อมูลออกจากกัน ช่วยให้แต่ละส่วนพัฒนาและทดสอบได้อิสระ แนวคิดย่อยที่ระบบนำมาใช้มีดังนี้

2.1.1 MVC และ REST Controller

รูปแบบ Model View Controller แยกการรับคำขอออกจากข้อมูลและการแสดงผล ในระบบนี้ Controller ของ Spring MVC รับคำขอ HTTP แล้วส่งข้อมูลกลับเป็น JSON ให้หน้าเว็บ React ซึ่งทำหน้าที่เป็นส่วนแสดงผลแยกต่างหาก

2.1.2 Service Layer

Service Layer เป็นขอบเขตของแอปพลิเคชันที่รวบรวมการทำงานทางธุรกิจ ควบคุม transaction และตรวจสิทธิ์ก่อนเข้าถึงข้อมูล ทำให้ Controller ไม่ต้องมีตรรกะทางธุรกิจ (Fowler, 2002)

2.1.3 Repository

Repository เป็นตัวกลางระหว่างโดเมนกับการเข้าถึงข้อมูล โดยทำงานคล้ายคอลเลกชันของวัตถุ ซ่อนรายละเอียดของคำสั่ง SQL ไว้ภายใน ระบบนี้ใช้ interface ของ Spring Data JPA ซึ่งสร้าง implementation ให้อัตโนมัติ

2.1.4 DTO และ Mapper

Data Transfer Object คือวัตถุที่ใช้ส่งข้อมูลข้ามขอบเขตของระบบ การใช้ DTO แทน Entity ป้องกันไม่ให้ข้อมูลภายใน เช่น รหัสผ่านที่เข้ารหัสแล้ว หลุดออกไปทาง API ส่วน Mapper ทำหน้าที่แปลงข้อมูลระหว่าง Entity กับ DTO

2.1.5 Dependency Injection

Dependency Injection คือการให้ภายนอกส่งวัตถุที่คลาสต้องใช้เข้ามา แทนการให้คลาสสร้างวัตถุนั้นเอง Spring Framework จัดการผ่าน IoC container ซึ่งสร้างและเชื่อม bean ให้ตามชนิดที่ประกาศไว้ ทำให้สลับ implementation และเขียนการทดสอบที่ใช้วัตถุจำลองได้ง่าย (Spring, n.d.-a)

2.2 หลักการ SOLID

Martin (2017) รวบรวมหลักการออกแบบเชิงวัตถุ 5 ข้อที่ช่วยให้ซอฟต์แวร์เปลี่ยนแปลงได้ง่าย เข้าใจง่าย และนำกลับมาใช้ใหม่ได้ เรียกรวมกันว่า SOLID ประกอบด้วย

2.2.1 Single Responsibility Principle

คลาสหนึ่งควรมีเหตุผลในการเปลี่ยนแปลงเพียงเรื่องเดียว หรือกล่าวอีกนัยหนึ่งคือรับผิดชอบต่อผู้มีส่วนได้ส่วนเสียกลุ่มเดียว

2.2.2 Open/Closed Principle

ซอฟต์แวร์ควรเปิดให้ขยายความสามารถได้ แต่ปิดไม่ให้ต้องแก้โค้ดเดิม การเพิ่มพฤติกรรมใหม่จึงควรทำผ่านการสร้างคลาสใหม่ที่ทำตาม interface เดิม

2.2.3 Liskov Substitution Principle

วัตถุของคลาสลูกต้องใช้แทนวัตถุของคลาสแม่หรือ interface ได้โดยพฤติกรรมของโปรแกรมยังถูกต้อง ซึ่งต้องอาศัยสัญญาของ input และ output ที่ชัดเจน

2.2.4 Interface Segregation Principle

ผู้ใช้งาน interface ไม่ควรถูกบังคับให้ขึ้นกับเมธอดที่ตนไม่ได้ใช้ จึงควรแยก interface ใหญ่ออกเป็น interface ย่อยตามกลุ่มการใช้งาน

2.2.5 Dependency Inversion Principle

โมดูลระดับสูงไม่ควรขึ้นกับโมดูลระดับต่ำโดยตรง ทั้งสองควรขึ้นกับนามธรรม เช่น interface และให้ระบบฉีด implementation ที่แท้จริงเข้ามาขณะทำงาน

2.3 รูปแบบการออกแบบของ GoF

Gamma, Helm, Johnson, และ Vlissides (1994) นำเสนอรูปแบบการออกแบบ 23 รูปแบบ แบ่งเป็น 3 กลุ่ม ได้แก่ Creational ว่าด้วยการสร้างวัตถุ Structural ว่าด้วยการประกอบคลาสและวัตถุ และ Behavioral ว่าด้วยการแบ่งหน้าที่และการสื่อสารระหว่างวัตถุ ระบบนี้เลือกกลุ่ม Behavioral เพราะปัญหาหลักของระบบเป็นเรื่องพฤติกรรม ได้แก่ การให้คะแนนคอร์ส วงจรชีวิตของคอร์ส และการแจ้งเหตุการณ์ โดยใช้ 3 รูปแบบดังนี้

2.3.1 Strategy Pattern

Strategy นิยามกลุ่มของอัลกอริทึมที่ใช้แทนกันได้ แต่ละอัลกอริทึมห่อหุ้มอยู่ในคลาสของตนเองภายใต้ interface เดียวกัน ผู้เรียกใช้จึงเลือกหรือเพิ่มอัลกอริทึมได้โดยไม่ต้องแก้โค้ดของผู้เรียก (Gamma et al., 1994)

2.3.2 State Pattern

State ให้วัตถุเปลี่ยนพฤติกรรมตามสถานะภายใน โดยแยกพฤติกรรมของแต่ละสถานะเป็นคลาสแทนการใช้คำสั่ง if หรือ switch จำนวนมากตรวจสถานะในทุกเมธอด (Gamma et al., 1994)

2.3.3 Observer Pattern

Observer กำหนดความสัมพันธ์แบบหนึ่งต่อหลาย เมื่อวัตถุต้นทางเปลี่ยนแปลง ผู้ติดตามทั้งหมดจะได้รับแจ้งโดยอัตโนมัติ ทำให้ต้นทางไม่ต้องรู้จักรายละเอียดของผู้ติดตาม (Gamma et al., 1994)

2.4 REST API และรหัสสถานะ HTTP

Fielding (2000) นิยาม REST เป็นรูปแบบสถาปัตยกรรมของระบบเครือข่ายที่มองทุกสิ่งเป็น resource ซึ่งระบุด้วย URI และจัดการผ่าน interface ที่เป็นแบบเดียวกัน ระบบนี้ออกแบบ endpoint ตาม resource เช่น

/api/v1/providers/{providerId}/courses และใช้เมธอด HTTP ตามความหมาย คือ GET สำหรับอ่าน POST สำหรับสร้าง PUT สำหรับแก้ไข และ DELETE สำหรับลบ พร้อมตอบรหัสสถานะตามความหมายดังตารางที่ 2.1

ตารางที่ 2.1 รหัสสถานะ HTTP ที่ระบบใช้

รหัส	ความหมาย	ตัวอย่างการใช้ในระบบ
200	OK	อ่านหรือแก้ไขข้อมูลสำเร็จ
201	Created	สร้างผู้ให้บริการหรือคอร์สสำเร็จ พร้อม header Location
204	No Content	ลบข้อมูลสำเร็จหรือออกจากระบบสำเร็จ
400	Bad Request	ข้อมูลไม่ผ่าน validation พร้อมรายการ fieldErrors
401	Unauthorized	ยังไม่เข้าสู่ระบบ session หมดอายุ หรือบัญชีถูกระงับ
403	Forbidden	ไม่มีสิทธิ์ หรือ CSRF token ไม่ถูกต้อง
404	Not Found	ไม่พบข้อมูล หรือข้อมูลที่ผู้ใช้ไม่ควรเห็น
409	Conflict	สถานะไม่อนุญาต ลบข้อมูลที่ยังถูกอ้างอิง หรือข้อมูลถูกแก้ไขชนกัน
500	Internal Server Error	ข้อผิดพลาดที่ไม่คาดคิดของเซิร์ฟเวอร์

2.5 ฐานข้อมูลเชิงสัมพันธ์ ORM และ Database Migration

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์สที่รองรับ foreign key, CHECK constraint, UNIQUE constraint, index และ transaction ตามมาตรฐาน SQL (The PostgreSQL Global Development Group, n.d.) ระบบเชื่อมต่อฐานข้อมูลผ่าน Spring Data JPA ซึ่งใช้ Hibernate เป็นตัวแปลงระหว่างวัตถุกับตาราง หรือ Object Relational Mapping

Database Migration คือการควบคุมเวอร์ชันของโครงสร้างฐานข้อมูลด้วยสคริปต์ที่เรียงลำดับ Flyway จะอ่านไฟล์ที่ตั้งชื่อตามรูปแบบ V1__ชื่อ.sql แล้วรันเฉพาะเวอร์ชันที่ยังไม่เคยรัน พร้อมบันทึกประวัติไว้ในตาราง flyway_schema_history ทำให้ทุกเครื่องได้ schema เดียวกัน (Redgate Software, n.d.) ระบบตั้งค่า Hibernate ให้ตรวจสอบเท่านั้นด้วย ddl-auto=validate เพื่อไม่ให้ schema ถูกแก้ไขโดยอัตโนมัติ

สำหรับการแก้ไขข้อมูลพร้อมกัน ระบบใช้ optimistic locking โดยเพิ่มคอลัมน์ version ในตารางที่มีการแก้ไขชนกันได้ เมื่อมีผู้ใช้สองคนแก้ข้อมูลชุดเดียวกัน ผู้ที่บันทึกทีหลังจะได้รับรหัส 409 และต้องโหลดข้อมูลใหม่ก่อนแก้ไขอีกครั้ง

2.6 ความปลอดภัยของเว็บแอปพลิเคชัน

Spring Security เป็นเฟรมเวิร์กด้านการยืนยันตัวตนและการกำหนดสิทธิ์ของ Spring (Spring, n.d.-b) ระบบใช้การยืนยันตัวตนแบบ session cookie ที่ตั้งค่า HttpOnly เพื่อไม่ให้ JavaScript อ่านค่าได้ ตั้งค่า Secure เพื่อส่งผ่าน HTTPS เท่านั้น และตั้งค่า SameSite=Lax เพื่อลดการส่ง cookie จากเว็บไซต์อื่น รหัสผ่านถูกเก็บเป็นค่า hash ด้วย BCrypt ซึ่งเป็นฟังก์ชันที่ออกแบบให้คำนวณช้าเพื่อต้านการเดารหัสผ่าน

Cross-Site Request Forgery หรือ CSRF คือการหลอกให้เบราว์เซอร์ของผู้ใช้ที่เข้าสู่ระบบแล้วส่งคำขอที่ผู้ใช้ไม่ได้ตั้งใจ OWASP Foundation (n.d.) แนะนำให้ใช้ token ที่ผู้โจมตีคาดเดาไม่ได้ ระบบจึงกำหนดให้ทุกคำขอที่เปลี่ยนแปลงข้อมูลต้องแนบ header X-XSRF-TOKEN ที่ได้จาก /api/v1/auth/csrf หากไม่ตรงกันระบบตอบ 403

2.7 Single Page Application และ React

Single Page Application คือเว็บแอปพลิเคชันที่โหลดหน้า HTML ครั้งเดียว แล้วเปลี่ยนเนื้อหาด้วย JavaScript ตามการใช้งาน ทำให้ตอบสนองได้รวดเร็ว React เป็นไลบรารีสำหรับสร้างส่วนติดต่อผู้ใช้จาก component ที่นำกลับมาใช้ซ้ำได้ (Meta Platforms, n.d.) ระบบใช้ React ร่วมกับ TypeScript เพื่อตรวจชนิดข้อมูล ใช้ Vite เป็นเครื่องมือ build ใช้ React Router จัดการเส้นทางของหน้า และใช้ Tailwind CSS จัดรูปแบบหน้าจอ

2.8 การทดสอบซอฟต์แวร์

Cohn (2009) เสนอแนวคิดพีระมิดการทดสอบ ซึ่งให้มีการทดสอบระดับหน่วยจำนวนมากที่สุดเพราะรันเร็วและระบุจุดผิดได้แม่นยำ รองลงมาคือการทดสอบระดับบริการ และมีการทดสอบผ่านส่วนติดต่อผู้ใช้จำนวนน้อยที่สุด ระบบนี้ทดสอบตามแนวคิดดังกล่าวในระดับต่าง ๆ ดังนี้

2.8.1 การทดสอบระดับหน่วย

ทดสอบกฎทางธุรกิจ การตรวจสิทธิ์ และการเปลี่ยนสถานะโดยแยกจากฐานข้อมูล ใช้ JUnit 5 เป็นเฟรมเวิร์กหลัก (The JUnit Team, n.d.) และใช้ Mockito สร้างวัตถุจำลองแทน repository

2.8.2 การทดสอบการทำงานร่วมกัน

ทดสอบ Controller, Service, Repository และ Spring Security ร่วมกันด้วย Spring Boot Test และ MockMvc โดยใช้ Testcontainers เปิดฐานข้อมูล PostgreSQL จริงใน container สำหรับการทดสอบแต่ละรอบ (Testcontainers, n.d.)

2.8.3 การทดสอบส่วนติดต่อผู้ใช้

ทดสอบการแสดงผล การกรอกแบบฟอร์ม และการตอบสนองต่อผลลัพธ์จาก API ด้วย Vitest ร่วมกับ React Testing Library

2.8.4 การทดสอบแบบครบวงจร

ทดสอบเส้นทางการใช้งานจริงผ่านเบราว์เซอร์ตั้งแต่หน้าเว็บจนถึงฐานข้อมูลด้วย Playwright บน Chromium (Microsoft, n.d.) และทดสอบการยอมรับของผู้ใช้บนระบบที่ให้บริการจริง

2.9 Container, CI/CD และบริการคลาวด์

Docker บรรจุแอปพลิเคชันพร้อมสภาพแวดล้อมไว้ใน container ที่ทำงานเหมือนกันทุกเครื่อง ส่วน Docker Compose ใช้ไฟล์ docker-compose.yml กำหนดหลายบริการให้เริ่มทำงานพร้อมกันด้วยคำสั่งเดียว (Docker, n.d.) GitHub Actions เป็นบริการ CI/CD ที่รันขั้นตอน build ทดสอบ และ deploy อัตโนมัติเมื่อมีการเปิด Pull Request หรือรวมงาน (GitHub, n.d.)

บริการคลาวด์ที่ระบบใช้มี 3 ส่วน Vercel ให้บริการไฟล์ของ Frontend และรองรับการตั้งค่า rewrite เพื่อส่งต่อคำขอบางเส้นทางไปยังเซิร์ฟเวอร์อื่น (Vercel, n.d.) Render ให้บริการ Backend จาก Dockerfile โดยบริการระดับใช้ฟรีจะหยุดพักเมื่อไม่มีคำขอเข้ามาเป็นเวลา 15 นาที และต้องใช้เวลาเริ่มทำงานใหม่ในคำขอถัดไป (Render, n.d.) และ Neon ให้บริการ PostgreSQL แบบ serverless ที่ลดทรัพยากรลงเมื่อไม่มีการใช้งาน (Neon, n.d.) ข้อจำกัดเหล่านี้มีผลต่อการออกแบบหน้าโหลดและการลองใหม่ของระบบ

บทที่ 3
การวิเคราะห์และออกแบบระบบ

บทนี้นำเสนอการวิเคราะห์ความต้องการ การออกแบบสถาปัตยกรรม ฐานข้อมูล API และการนำรูปแบบการออกแบบมาใช้ในระบบ CourseHub แผนภาพทั้งหมดอ้างอิงจากเอกสารในโฟลเดอร์ doc/diagrams ของโครงงาน (กลุ่ม 36, 2569)

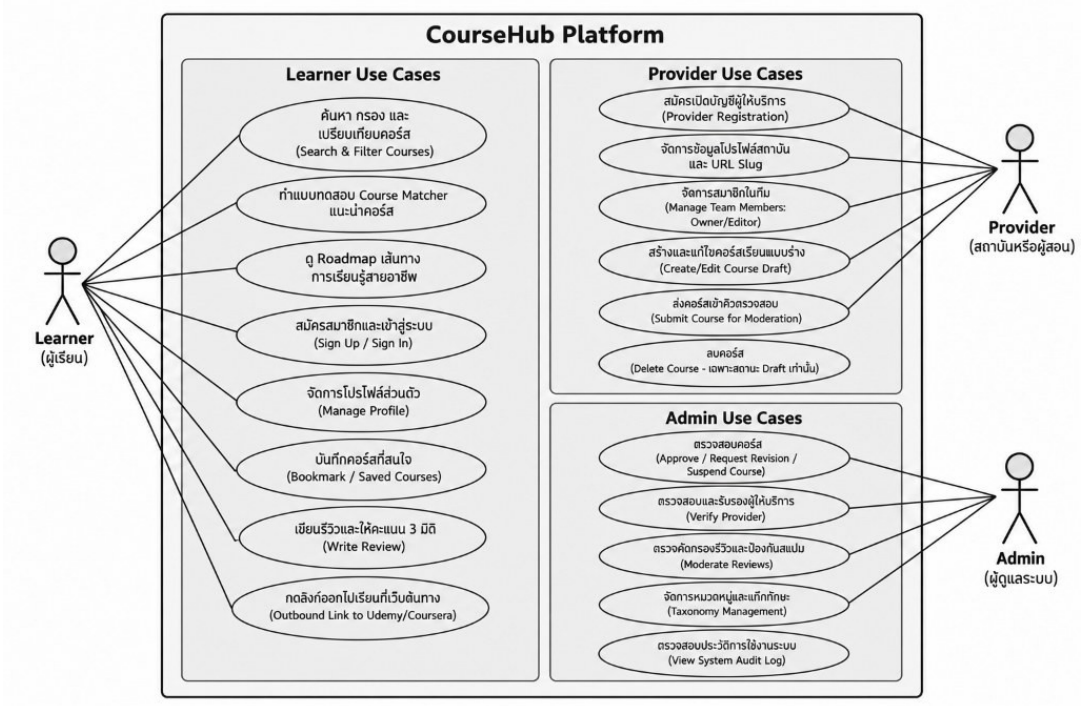
3.1 การวิเคราะห์ความต้องการของระบบ

3.1.1 ผู้ใช้งานระบบ

ระบบมีผู้กระทำ 3 กลุ่ม ผู้เรียนใช้งานส่วนค้นหาและแนะนำคอร์สได้โดยไม่ต้องเข้าสู่ระบบ แต่ต้องเข้าสู่ระบบก่อนบันทึกคอร์สหรือเขียนรีวิว ผู้ให้บริการคือผู้ใช้ที่สมัครเปิดบัญชีสถาบัน โดยผู้สมัครได้บทบาท Owner ทันที และ Owner เพิ่มสมาชิกเป็น Owner หรือ Editor ได้ ส่วนผู้ดูแลระบบคือบัญชีบทบาท ADMIN ที่ตรวจสอบข้อมูลก่อนเผยแพร่

3.1.2 แผนภาพกรณีการใช้งาน

แผนภาพกรณีการใช้งานในภาพที่ 3.1 แสดงความสัมพันธ์ระหว่างผู้กระทำทั้ง 3 กลุ่มกับกรณีการใช้งาน 19 กรณี แบ่งเป็นกรณีของผู้เรียน 8 กรณี ผู้ให้บริการ 6 กรณี และผู้ดูแลระบบ 5 กรณี



ภาพที่ 3.1 แผนภาพกรณีการใช้งานของระบบ CourseHub

3.1.3 รายการกรณีการใช้งาน

รายการกรณีการใช้งานและสถานะการพัฒนาแสดงในตารางที่ 3.1 กรณีที่ยังไม่พัฒนามี 2 กรณี คือ UC03 และ UC18 ซึ่งอยู่ในแผนพัฒนาต่อ

ตารางที่ 3.1 รายการกรณีการใช้งานและสถานะการพัฒนา

รหัส	กรณีการใช้งาน	ผู้กระทำ	สถานะ
UC01	ค้นหา กรอง และเปรียบเทียบคอร์ส	ผู้เรียน	พัฒนาแล้ว
UC02	ทำแบบทดสอบ Matcher แนะนำคอร์ส	ผู้เรียน	พัฒนาแล้ว
UC03	ดูเส้นทางการเรียนรู้สายอาชีพ	ผู้เรียน	แผนพัฒนาต่อ
UC04	สมัครสมาชิกและเข้าสู่ระบบ	ผู้เรียน	พัฒนาแล้ว
UC05	จัดการโปรไฟล์ส่วนตัว	ผู้เรียน	พัฒนาแล้ว
UC06	บันทึกคอร์สที่สนใจ	ผู้เรียน	พัฒนาแล้ว
UC07	เขียนรีวิวและให้คะแนน	ผู้เรียน	พัฒนาแล้ว
UC08	กดลิงก์ไปเรียนที่เว็บไซต์ต้นทาง	ผู้เรียน	พัฒนาแล้ว
UC09	สมัครเปิดบัญชีผู้ให้บริการ	ผู้ให้บริการ	พัฒนาแล้ว
UC10	จัดการข้อมูลสถาบันและ URL Slug	ผู้ให้บริการ	พัฒนาแล้ว
UC11	จัดการสมาชิกในทีม	ผู้ให้บริการ	พัฒนาแล้ว
UC12	สร้างและแก้ไขคอร์สแบบร่าง	ผู้ให้บริการ	พัฒนาแล้ว
UC13	ส่งคอร์สเข้าคิวตรวจสอบ	ผู้ให้บริการ	พัฒนาแล้ว
UC14	ลบคอร์สแบบร่าง	ผู้ให้บริการ	พัฒนาแล้ว
UC15	ตรวจสอบและอนุมัติคอร์ส	ผู้ดูแลระบบ	พัฒนาแล้ว
UC16	ตรวจสอบและรับรองผู้ให้บริการ	ผู้ดูแลระบบ	พัฒนาแล้ว
UC17	ตรวจคัดกรองรีวิว	ผู้ดูแลระบบ	พัฒนาแล้ว
UC18	จัดการหมวดหมู่และทักษะ	ผู้ดูแลระบบ	แผนพัฒนาต่อ
UC19	ตรวจสอบประวัติการใช้งานระบบ	ผู้ดูแลระบบ	พัฒนาแล้ว

3.1.4 คำอธิบายกรณีการใช้งานหลัก

3.1.4.1 UC02 ทำแบบทดสอบ Matcher แนะนำคอร์ส

ผู้เรียนเลือกหมวดหมู่เป้าหมาย ระดับพื้นฐาน ภาษา งบประมาณเป็นบาท และเวลาเรียนต่อสัปดาห์ แล้วส่งคำตอบไปยัง POST /api/v1/course-matches ระบบเลือกเฉพาะคอร์สที่เผยแพร่แล้วจากผู้ให้บริการที่ได้รับการรับรอง กรองหมวดหมู่ ระดับ ภาษา และงบประมาณก่อน จากนั้นคำนวณคะแนน3 ด้านแล้วส่งคืนคอร์สไม่เกิน 3 อันดับพร้อมเหตุผล หากไม่มีคอร์สผ่านเงื่อนไข ระบบส่งรายการว่างพร้อมข้อจำกัดโดยไม่ผ่อนเงื่อนไข

3.1.4.2 UC12 และ UC13 สร้างคอร์สแบบร่างและส่งตรวจ

สมาชิกทีมบทบาท Owner หรือ Editor กรอกชื่อคอร์ส คำอธิบาย ลิงก์ต้นทาง ระดับภาษา ระยะเวลาเรียน ราคา และหมวดหมู่ ระบบบันทึกคอร์สในสถานะ DRAFT และระบุแพลตฟอร์มจากโดเมนของ URL โดยอัตโนมัติ เมื่อพร้อมแล้วผู้ให้บริการกดส่งตรวจ ระบบเปลี่ยนสถานะเป็น PENDING ได้เฉพาะเมื่อผู้ให้บริการมีสถานะ ACTIVE หากยังไม่ได้รับการรับรอง ระบบตอบ 409

3.1.4.3 UC15 ตรวจสอบและอนุมัติคอร์ส

ผู้ดูแลระบบเปิดรายการคอร์สที่รอตรวจ แล้วเลือกอนุมัติ ขอให้แก้ไข ระงับ คืนสถานะหรือเก็บถาวร โดยการขอแก้ไข ระงับ และเก็บถาวรต้องระบุเหตุผล ระบบเปลี่ยนสถานะตามกติกาของ State Pattern บันทึกการกระทำลง audit_logs ใน transaction เดียวกัน และส่งเหตุการณ์ให้ Observer นับสถิติหลัง commit สำเร็จ

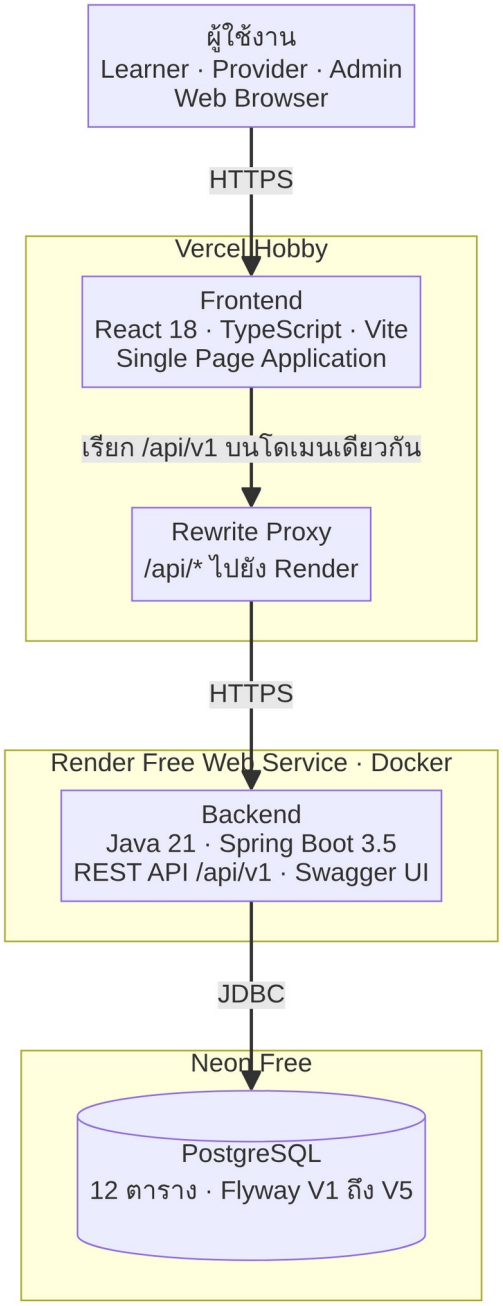
3.1.5 ความต้องการที่ไม่ใช่ฟังก์ชัน

1. ด้านความปลอดภัย ระบบตรวจสิทธิ์ที่เซิร์ฟเวอร์ทุกครั้งโดยไม่พึ่งการซ่อนปุ่มบนหน้าเว็บ และ API ต้องไม่ส่งรหัสผ่าน ค่า hash หรือ token ออกไป
2. ด้านความถูกต้องของข้อมูล การเปลี่ยนสถานะกับการเขียนประวัติต้องสำเร็จพร้อมกันหรือยกเลิกพร้อมกัน และการแก้ไขชนกันต้องถูกตรวจพบ
3. ด้านการใช้งาน หน้าเว็บต้องแสดงสถานะกำลังเตรียมระบบและปุ่มลองใหม่ระหว่างที่ Backend เริ่มทำงาน แทนการแสดงหน้าว่างหรือข้อผิดพลาดที่ทำให้เข้าใจผิด
4. ด้านการบำรุงรักษา โค้ดต้องแยกชั้นชัดเจน โครงสร้างฐานข้อมูลเปลี่ยนผ่าน migration เท่านั้น และมีการทดสอบอัตโนมัติก่อนนำขึ้นระบบจริง

3.2 การออกแบบสถาปัตยกรรมระบบ

3.2.1 สถาปัตยกรรมการติดตั้งระบบ

ระบบแบ่งการติดตั้งเป็น 3 ส่วนบนผู้ให้บริการคลาวด์ต่างกัน ดังภาพที่ 3.2 เบราว์เซอร์เรียกหน้าเว็บจาก Vercel และเรียก API ผ่านเส้นทาง /api บนโดเมนเดียวกัน จากนั้น Vercel rewrite ส่งต่อคำขอไปยัง Backend บน Render ซึ่งเชื่อมต่อฐานข้อมูล PostgreSQL บน Neon ผ่าน JDBC การออกแบบให้เบราว์เซอร์เห็นเพียงโดเมนเดียวทำให้ session cookie และ CSRF token ทำงานได้โดยไม่ต้องเปิด CORS ข้ามโดเมน

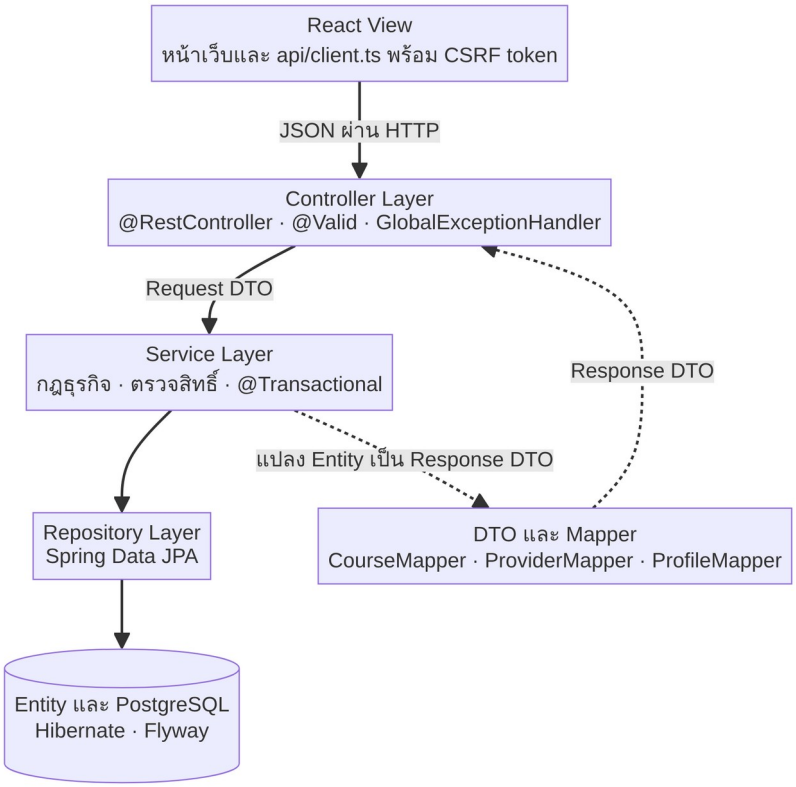


ภาพที่ 3.2 สถาปัตยกรรมการติดตั้งระบบบนคลาวด์

3.2.2 สถาปัตยกรรมแบบหลายชั้นของ Backend

Backend แบ่งชั้นตามภาพที่ 3.3 Controller รับคำขอและตรวจข้อมูลด้วย @Valid แล้วส่งต่อให้ Service ซึ่งบรรจุกฎธุรกิจ การตรวจสิทธิ์ และการควบคุม transaction ส่วน Repository เข้าถึงฐานข้อมูลผ่าน Spring Data JPA ทุกเส้นทางต้องผ่านลำดับ Controller ไป Service ไป Repository โดยไม่มี Controller ใดเรียก

Repository โดยตรง ตัวอย่างเช่น CourseController เรียก CourseCommandService ซึ่ง implement โดย CourseServiceImpl แล้วจึงเรียก CourseRepository ข้อผิดพลาดทั้งหมดถูกจัดการที่ GlobalExceptionHandler และตอบกลับในรูปแบบเดียวกัน คือ {timestamp, status, code, message, path, fieldErrors}



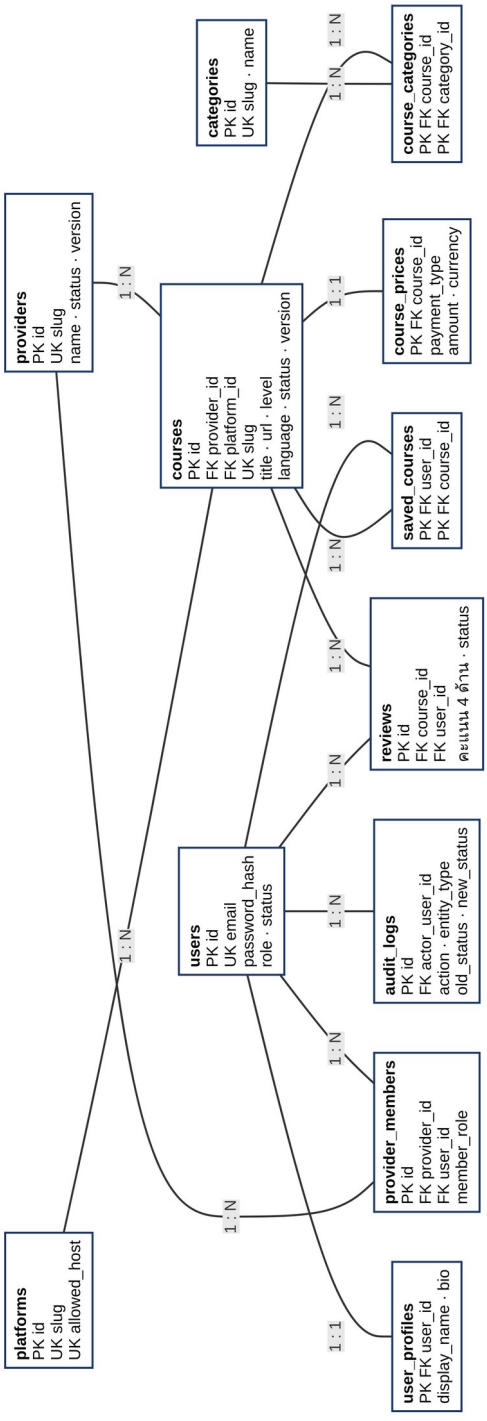
ภาพที่ 3.3 สถาปัตยกรรมแบบหลายชั้นของ Backend

โค้ด Backend จัดกลุ่ม package ตามโดเมน ได้แก่ auth, profile, provider, course, catalog, bookmark, review, matcher และ audit โดยมี package ส่วนกลาง ได้แก่ security, exception, common และ config ภายในแต่ละโดเมนมี Controller, Service, Repository, DTO และ Mapper ของตนเอง ทำให้สมาชิกแต่ละคนพัฒนาโดเมนที่รับผิดชอบได้โดยกระทบส่วนอื่นน้อย

3.3 การออกแบบฐานข้อมูล

3.3.1 แผนภาพความสัมพันธ์ของข้อมูล

ฐานข้อมูลประกอบด้วย 12 ตาราง ความสัมพันธ์ระหว่างตารางแสดงในภาพที่ 3.4 และคำอธิบายของแต่ละตารางแสดงในตารางที่ 3.2



ภาพที่ 3.4 แผนภาพความสัมพันธ์ของข้อมูลในระบบ CourseHub

ตารางที่ 3.2 ตารางในฐานข้อมูลและหน้าที่

ตาราง	หน้าที่
users	บัญชีผู้ใช้ อีเมล รหัสผ่านแบบ hash บทบาท LEARNER หรือ ADMIN และสถานะบัญชี
user_profiles	ชื่อที่แสดงและประวัติย่อ เชื่อมกับ users แบบหนึ่งต่อหนึ่ง
providers	ข้อมูลสถาบันผู้ให้บริการ slug และสถานะ PENDING ACTIVE หรือ SUSPENDED
provider_members	สมาชิกของผู้ให้บริการและบทบาท OWNER หรือ EDITOR
platforms	แพลตฟอร์มต้นทางและโดเมนที่อนุญาต
courses	ข้อมูลคอร์ส ลิงก์ ระดับ ภาษา ระยะเวลาเรียน สถานะ และเหตุผลการตรวจล่าสุด
course_prices	รูปแบบราคา FREE ONE_TIME หรือ SUBSCRIPTION จำนวนเงิน และสกุลเงิน
categories	หมวดหมู่ของคอร์ส
course_categories	ตารางเชื่อมความสัมพันธ์แบบหลายต่อหลายระหว่างคอร์สกับหมวดหมู่
reviews	รีวิวและคะแนน 4 ด้าน สถานะ PENDING PUBLISHED หรือ REJECTED
saved_courses	คอร์สที่ผู้เรียนบันทึกไว้
audit_logs	ประวัติการกระทำ ผู้กระทำ ชนิดข้อมูล สถานะก่อนและหลัง เหตุผล และเวลา

3.3.2 ความสัมพันธ์และข้อจำกัดของข้อมูล

ความสัมพันธ์แบบหนึ่งต่อหนึ่งมี 2 คู่ ได้แก่ users กับ user_profiles และ courses กับ course_prices ทั้งสองคู่ใช้ primary key ร่วมกัน โดย Entity ใช้ @MapsId เพื่อบังคับให้ผู้ใช้หนึ่งคนมีโปรไฟล์ได้เพียงหนึ่งรายการ ความสัมพันธ์แบบหนึ่งต่อหลาย เช่น providers กับ courses และ courses กับ reviews ส่วนความสัมพันธ์แบบหลายต่อหลายระหว่าง courses กับ categories ใช้ตารางเชื่อม course_categories

foreign key จาก courses ไปยัง providers และ platforms กำหนดเป็น ON DELETE RESTRICT จึงลบผู้ให้บริการที่ยังมีคอร์สไม่ได้ ขณะที่ข้อมูลที่เป็นส่วนประกอบของคอร์ส เช่น ราคาและการเชื่อมหมวดหมู่ กำหนดเป็น ON DELETE CASCADE เพื่อให้ถูกลบไปพร้อมคอร์ส ตาราง reviews มี UNIQUE บนคู่ course_id และ user_id เพื่อให้ผู้ใช้หนึ่งคนมีรีวิวต่อคอร์สได้เพียงรายการเดียว และมี CHECK บังคับคะแนนอยู่ในช่วง 1 ถึง 5 ตาราง providers, courses และ reviews มีคอลัมน์ version สำหรับ optimistic locking และความสัมพันธ์ @ManyToOne ใช้ fetch แบบ LAZY เพื่อไม่ให้โหลดข้อมูลที่ไม่จำเป็น

ระบบสร้าง index สำหรับคำค้นที่ใช้บ่อย เช่น idx_courses_catalog บนคอลัมน์ status, created_at และ id สำหรับหน้าค้นหาคอร์ส idx_reviews_course_status สำหรับคำนวณคะแนนเฉลี่ย และ idx_audit_logs_lookup สำหรับค้นประวัติตามชนิดและรหัสข้อมูล

3.3.3 การควบคุมเวอร์ชันของฐานข้อมูล

โครงสร้างฐานข้อมูลทั้งหมดสร้างผ่าน Flyway จำนวน 5 เวอร์ชัน ดังตารางที่ 3.3

ตารางที่ 3.3 ไฟล์ migration ของ Flyway

ไฟล์	การเปลี่ยนแปลง
V1__init_schema.sql	สร้าง 12 ตาราง พร้อม foreign key, CHECK, UNIQUE และ index
V2__seed_initial_data.sql	ข้อมูลตั้งต้น ได้แก่ แพลตฟอร์ม 5 แห่ง ผู้ให้บริการ 5 ราย หมวดหมู่ 6 หมวด และคอร์สตัวอย่าง 6 คอร์ส
V3__audit_log_reason.sql	เพิ่มคอลัมน์เหตุผลใน audit_logs และเหตุผลการตรวจล่าสุดใน courses
V4__review_moderation.sql	เพิ่ม version เหตุผลการตรวจ และ index สำหรับการตรวจรีวิว
V5__automatic_course_platforms.sql	ขยายความยาว URL ของคอร์สเป็น 2048 ตัวอักษร และบังคับโดเมนของแพลตฟอร์มไม่ให้ซ้ำ

3.4 การออกแบบ API

API ทั้งหมดขึ้นต้นด้วย /api/v1 และมีเอกสาร OpenAPI ที่เปิดดูผ่าน Swagger UI ได้ endpoint หลักแสดงในตารางที่ 3.4 รายการคอร์สใน GET /api/v1/courses รองรับการแบ่งหน้าและการเรียงลำดับผ่านพารามิเตอร์ page, size และ sort

ตารางที่ 3.4 endpoint หลักของระบบ

เมธอด	Endpoint	หน้าที่และสิทธิ์
POST	/api/v1/auth/register, /auth/login, /auth/logout	สมัครสมาชิก เข้าสู่ระบบ และออกจากระบบ
GET, PUT	/api/v1/me/profile	อ่านและแก้ไขโปรไฟล์ของผู้ใช้ที่เข้าสู่ระบบ
GET	/api/v1/courses?page&size&sort	ค้นหาคอร์สที่เผยแพร่ พร้อมการกรอง แบ่งหน้า และเรียงลำดับ

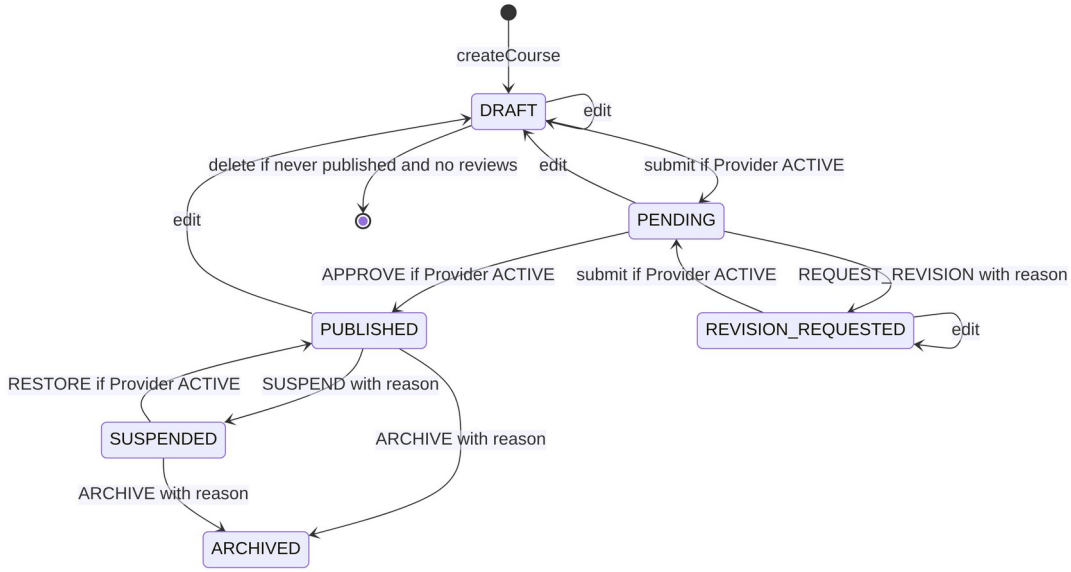
PUT, DELETE	/api/v1/me/bookmarks/{courseId}	บันทึกและยกเลิกการบันทึกคอร์ส ทำซ้ำได้ผลเหมือนเดิม
POST, PUT	/api/v1/courses/{courseId}/reviews	เขียนและแก้ไขรีวิว รีวิวกลับไปรอตรวจทุกครั้ง
POST	/api/v1/course-matches	แนะนำคอร์สตามคำตอบของแบบทดสอบ ทุกคนเรียกได้
POST	/api/v1/providers	สมัครเป็นผู้ให้บริการ ผู้สมัครได้บทบาท OWNER
GET, PUT, DELETE	/api/v1/providers/{id}	อ่าน แก้ไข และลบข้อมูลสถาบัน ลบได้เฉพาะ OWNER เมื่อไม่มีคอร์ส
GET, POST, DELETE	/api/v1/providers/{providerId}/members	จัดการสมาชิกทีม เฉพาะ OWNER
POST, GET	/api/v1/providers/{providerId}/courses	สร้างคอร์สแบบร่างและดูคอร์สของสถาบัน
PUT, DELETE	/api/v1/courses/{id}	แก้ไขคอร์ส และลบคอร์สแบบร่างที่ไม่เคยเผยแพร่
POST	/api/v1/courses/{id}/submissions	ส่งคอร์สให้ผู้ดูแลตรวจ
POST	/api/v1/admin/courses/{id}/moderation-decisions	ผู้ดูแลตัดสินผลการตรวจคอร์ส
POST	/api/v1/admin/providers/{id}/verification-decisions	ผู้ดูแลรับรองหรือระงับผู้ให้บริการ
GET, POST	/api/v1/admin/reviews	ผู้ดูแลดูคิวรีวิวและตัดสินผลการตรวจรีวิว
GET	/api/v1/admin/audit-logs	ผู้ดูแลดูประวัติการใช้งานพร้อมตัวกรองและการแบ่งหน้า

3.5 การประยุกต์ใช้รูปแบบการออกแบบ

3.5.1 State Pattern กับวงจรชีวิตของคอร์ส

คอร์สมี 6 สถานะ และแต่ละสถานะอนุญาตคำสั่งต่างกัน หากเขียนด้วยเงื่อนไขตรวจสถานะในทุกเมธอด โค้ดจะยาวและแก้ไขผิดได้ง่าย ระบบจึงสร้าง interface CourseWorkflowState และคลาสสถานะ 6 คลาส โดย CourseWorkflow ทำหน้าที่เลือกวัตถุสถานะจากค่า enum ที่เก็บในฐานข้อมูล แล้วให้วัตถุนั้นตัดสินว่าคำสั่งใด

ทำได้ Service ตรวจสิทธิ์ก่อนเสมอ และหากคำสั่งไม่อนุญาตในสถานะนั้น ระบบตอบ 409 การเปลี่ยนสถานะทั้งหมดแสดงในภาพที่ 3.5 และสรุปคำสั่งที่อนุญาตในตารางที่ 3.5



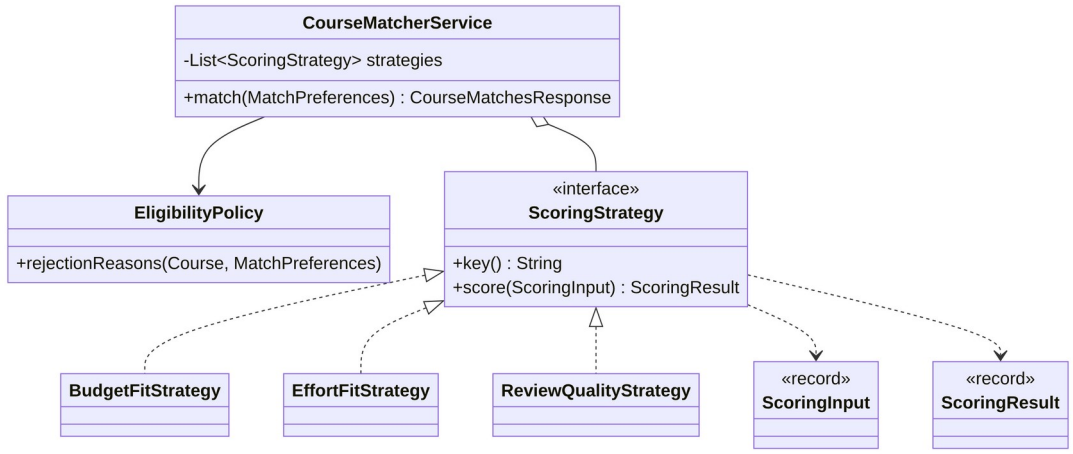
ภาพที่ 3.5 แผนภาพสถานะของคอร์ส

ตารางที่ 3.5 คำสั่งที่อนุญาตในแต่ละสถานะของคอร์ส

สถานะ	แก้ไข	ส่งตรวจ	ผู้ดูแลตัดสิน	ลบ
DRAFT	ได้ คงเป็น DRAFT	ได้ เมื่อผู้ให้บริการ ACTIVE	ไม่ได้	ได้ เมื่อไม่เคยเผยแพร่และไม่มีรีวิว
PENDING	ได้ กลับเป็น DRAFT	ไม่ได้	APPROVE หรือ REQUEST_REVISION	ไม่ได้
REVISION_REQUESTED	ได้ คงสถานะเดิม	ได้ เมื่อผู้ให้บริการ ACTIVE	ไม่ได้	ไม่ได้
PUBLISHED	ได้ กลับเป็น DRAFT	ไม่ได้	SUSPEND หรือ ARCHIVE	ไม่ได้
SUSPENDED	ไม่ได้	ไม่ได้	RESTORE หรือ ARCHIVE	ไม่ได้
ARCHIVED	ไม่ได้	ไม่ได้	ไม่ได้	ไม่ได้

3.5.2 Strategy Pattern กับการแนะนำคอร์ส

ผู้เรียนแต่ละคนให้ความสำคัญกับปัจจัยต่างกัน และในอนาคตอาจต้องเพิ่มสูตรคะแนนใหม่ ระบบจึงกำหนด interface ScoringStrategy ที่มีเมธอด key และ score ซึ่งรับ ScoringInput และคืน ScoringResult ที่บังคับคะแนนอยู่ในช่วง 0 ถึง 100 พร้อมเหตุผล คลาส CourseMatcherService รับรายการ ScoringStrategy ทั้งหมดผ่าน constructor แล้วเรียกทุกตัวในลูปเดียว ดังภาพที่ 3.6 การเพิ่มสูตรใหม่จึงทำได้ด้วยการเพิ่ม bean ที่ implement interface นี้โดยไม่ต้องแก้ CourseMatcherService



ภาพที่ 3.6 แผนภาพคลาสของ Strategy Pattern ในส่วน Course Matcher

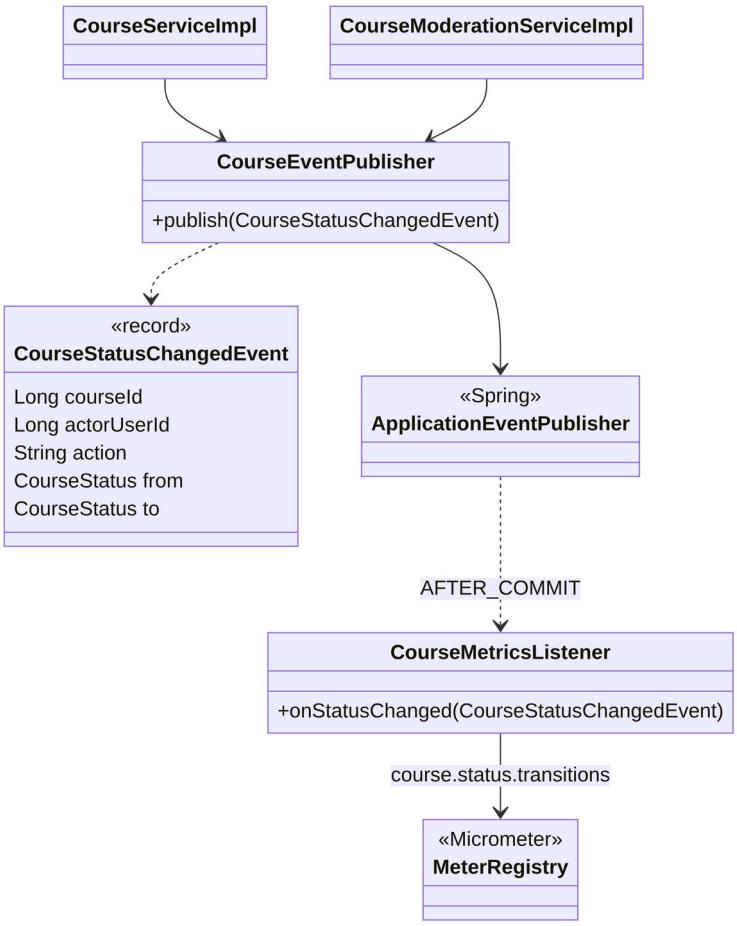
ก่อนคำนวณคะแนน EligibilityPolicy จะกรองคอร์สที่ไม่ตรงหมวดหมู่ ระดับ หรือภาษา และคอร์สที่ราคาเกินงบ คอร์สแบบ SUBSCRIPTION และคอร์สที่ราคาไม่ใช่สกุลบาทจะไม่ผ่าน เพราะยืนยันค่าใช้จ่ายรวมไม่ได้ จากนั้นจึงคำนวณคะแนนด้วยสูตรในตารางที่ 3.6 แล้วเฉลี่ยคะแนนทุกด้านด้วยน้ำหนักเท่ากัน เรียงจากมากไปน้อย และเลือก 3 อันดับแรก

ตารางที่ 3.6 สูตรคะแนนของแต่ละ Strategy

Strategy	สูตรคะแนน	กรณีไม่มีข้อมูล
BudgetFitStrategy	คอร์สฟรีได้ 100 คอร์สที่มีราคาได้ 100 × [1 − ราคา ÷ งบประมาณ]	ราคาที่ไม่ทราบถูกกรองออกก่อน
EffortFitStrategy	เวลาที่มี = ชั่วโมงต่อสัปดาห์ × 4 ถ้าเพียงพอได้ 100 ถ้าไม่พอได้ 100 × เวลาที่มี ÷ ระยะเวลาคอร์ส	ไม่ทราบระยะเวลาได้ 50
ReviewQualityStrategy	คะแนนรีวิวเฉลี่ย ÷ 5 × 100 นับเฉพาะรีวิวที่เผยแพร่	ไม่มีรีวิวได้ 50

3.5.3 Observer Pattern กับการติดตามการเปลี่ยนสถานะ

เมื่อคอร์สเปลี่ยนสถานะ ระบบต้องนับสถิติการเปลี่ยนสถานะ แต่ไม่ต้องการให้ Service หลักผูกติดกับระบบสถิติ ระบบจึงให้ CourseServiceImpl และ CourseModerationServiceImpl ส่ง CourseStatusChangedEvent ผ่าน CourseEventPublisher ซึ่งห่อ ApplicationEventPublisher ของ Spring ส่วน CourseMetricsListener รับเหตุการณ์ในจังหวะ AFTER_COMMIT แล้วเพิ่มค่า counter course.status.transitions ของ Micrometer แยกตาม action สถานะต้นทาง และสถานะปลายทาง ดังภาพที่ 3.7



ภาพที่ 3.7 แผนภาพคลาสของ Observer Pattern

การออกแบบนี้แยกข้อมูลถาวรออกจากสถิติ Audit Log ถูกเขียนใน transaction เดียวกับการเปลี่ยนสถานะ หากเขียนไม่สำเร็จ ธุรกรรมทั้งหมดจะถูกยกเลิก ขณะที่ Observer นับสถิติหลัง commit สำเร็จเท่านั้น หากการนับสถิติผิดพลาด ระบบบันทึก error log แต่ผู้ใช้ยังได้รับผลสำเร็จตามเดิม

3.5.4 การประยุกต์ใช้หลักการ SOLID

คณะผู้จัดทำวิเคราะห์หลักการ SOLID จากโค้ดจริงและบันทึกไว้ในเอกสาร doc/solid-analysis.md ตัวอย่างสรุปแสดงในตารางที่ 3.7

ตารางที่ 3.7 ตัวอย่างการประยุกต์ใช้หลักการ SOLID ในระบบ

หลักการ	ตัวอย่างในระบบ
Single Responsibility	CourseUrlPolicy รับผิดชอบกฎการตรวจ URL และ CourseMapper รับผิดชอบการแปลง Entity เป็น DTO แยกออกจาก CourseServiceImpl ที่ควบคุมลำดับการทำงาน
Open/Closed	เพิ่ม ScoringStrategy ใหม่ได้โดยไม่แก้ลูปจัดอันดับใน CourseMatcherService
Liskov Substitution	Strategy ทั้ง 3 ตัวผ่านชุดทดสอบสัญญาเดียวกันใน ScoringStrategyContractTests คือให้ผลคงที่ คะแนนอยู่ในช่วง 0 ถึง 100 และมีเหตุผลเสมอ
Interface Segregation	แยก CourseQueryService สำหรับการอ่านออกจาก CourseCommandService สำหรับการสร้าง แก้ไข ส่งตรวจ และลบ
Dependency Inversion	ProviderController ขึ้นกับ interface ProviderService และ CourseModerationServiceImpl ขึ้นกับ AdminActorResolver โดย Spring ฉีด implementation ผ่าน constructor

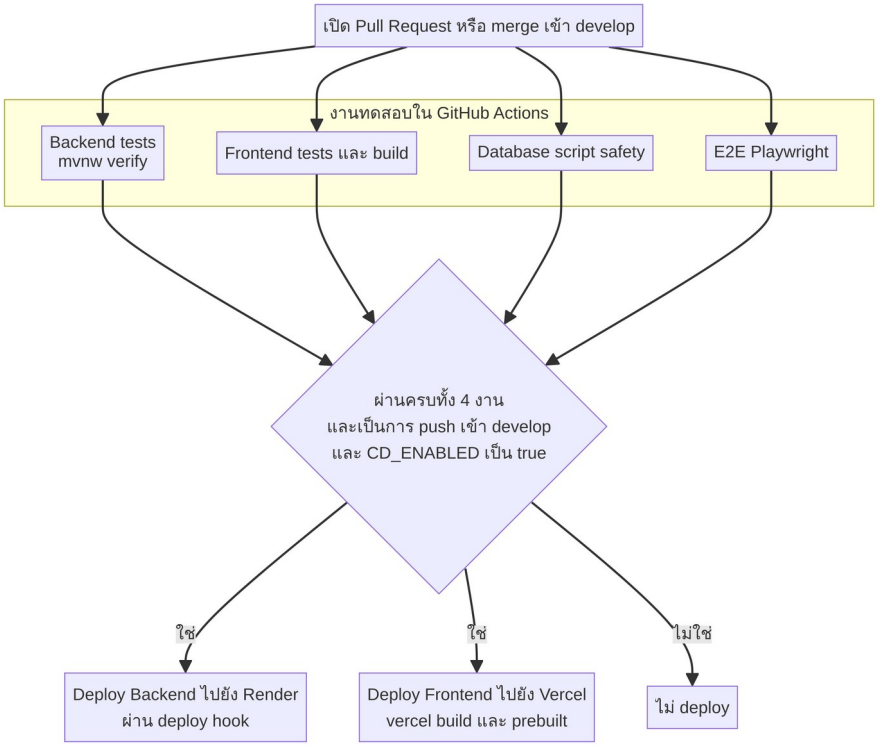
3.6 การออกแบบความปลอดภัยและสิทธิ์การเข้าถึง

ระบบกำหนดสิทธิ์ 2 ระดับ ระดับแรกคือบทบาทของบัญชี ได้แก่ LEARNER และ ADMIN ซึ่ง Spring Security ตรวจที่เส้นทาง /api/v1/admin ระดับที่สองคือบทบาทภายในผู้ให้บริการ ได้แก่ OWNER และ EDITOR ซึ่ง Service ตรวจความเป็นเจ้าของทุกครั้งก่อนแก้ข้อมูล ผู้ใช้จึงแก้ได้เฉพาะผู้ให้บริการและคอร์สของทีมตนเอง และการจัดการสมาชิกทำได้เฉพาะ OWNER โดยระบบไม่อนุญาตให้ลบ OWNER คนสุดท้าย

ข้อมูลที่ผู้ใช้ไม่มีสิทธิ์เห็น เช่น คอร์สที่ยังไม่เผยแพร่ของผู้ให้บริการอื่น ระบบตอบ 404 แทน 403 เพื่อไม่เปิดเผยว่ามีข้อมูลนั้นอยู่ บัญชีที่ถูกระงับจะถูกตัด session ทันทีและได้รับ 401 และ response ที่มีข้อมูลส่วนบุคคลกำหนด Cache-Control: no-store เพื่อไม่ให้ถูกเก็บในแคช

3.7 การออกแบบกระบวนการ CI/CD

กระบวนการ CI/CD ของระบบกำหนดไว้ในไฟล์ .github/workflows/ci.yml ดังภาพที่ 3.8 ทุก Pull Request และทุกการรวมงานเข้า develop จะรันงานทดสอบ 4 งาน ได้แก่ การทดสอบ Backend การทดสอบและ build Frontend การทดสอบความปลอดภัยของสคริปต์ฐานข้อมูล และการทดสอบ E2E บนฐานข้อมูลแยก เมื่อผ่านครบทั้ง 4 งาน และเป็นการ push เข้า develop ที่เปิดตัวแปร CD_ENABLED ไว้ ระบบจึง deploy Backend ไปยัง Render ผ่าน deploy hook และ deploy Frontend ไปยัง Vercel หากการทดสอบใดไม่ผ่าน ระบบจะไม่ deploy



ภาพที่ 3.8 กระบวนการ CI/CD ด้วย GitHub Actions

บทที่ 4
ผลการพัฒนาและการทดสอบระบบ

บทนี้นำเสนอผลการพัฒนาระบบ CourseHub ที่ให้บริการจริงบนคลาวด์ ผลการทดสอบในแต่ละระดับ ผลของกระบวนการ CI/CD และผลการทำงานร่วมกันของทีม ข้อมูลทั้งหมดอ้างอิงจากรายงานผลทดสอบในโฟลเดอร์ doc/test-reports ของโครงงาน (กลุ่ม 36, 2569)

4.1 ผลการพัฒนาระบบ

ระบบให้บริการหน้าเว็บที่ https://group36-coursehub.vercel.app และให้บริการเอกสาร API ที่ https://coursehub-backend-ahz2.onrender.com/swagger-ui.html ภาพหน้าจอในหัวข้อนี้บันทึกจากระบบที่ให้บริการจริงและจากหลักฐานการทดสอบการยอมรับ

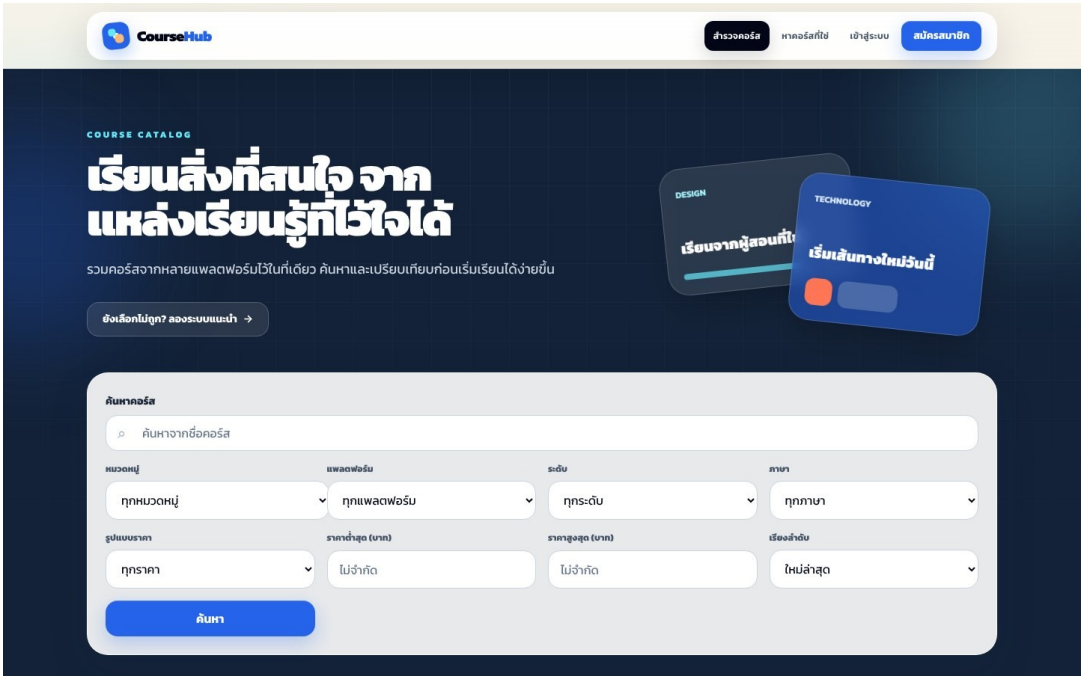
4.1.1 ส่วนของผู้เรียน

หน้าแรกของระบบในภาพที่ 4.1 แนะนำแนวคิดของระบบและมีปุ่มไปยังแบบทดสอบหาคอร์สที่ใช่และหน้าสำรวจคอร์สทั้งหมด



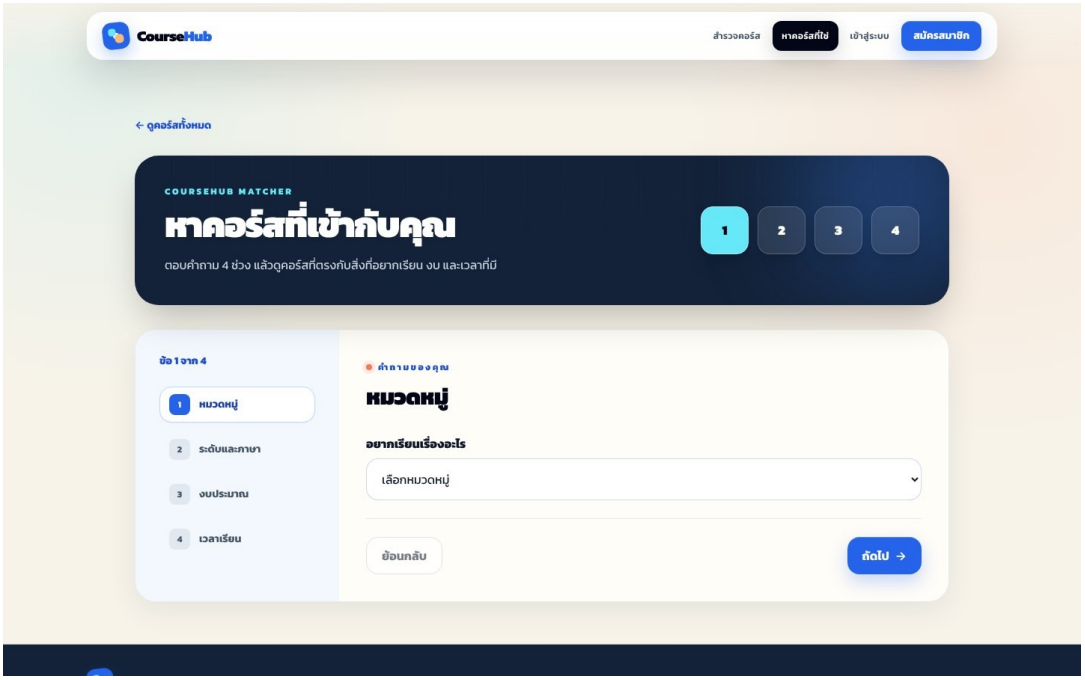
ภาพที่ 4.1 หน้าแรกของระบบ CourseHub

หน้าสำรวจคอร์สในภาพที่ 4.2 ให้ผู้เรียนค้นหาจากชื่อคอร์ส และกรองตามหมวดหมู่ แพลตฟอร์ม ระดับ ภาษา รูปแบบราคา และช่วงราคา พร้อมเลือกการเรียงลำดับ ผลลัพธ์แบ่งหน้าที่ฝั่งเซิร์ฟเวอร์ และการ์ดของแต่ละคอร์สแสดงแพลตฟอร์ม ระดับ ภาษา ระยะเวลา หมวดหมู่ คะแนนรีวิว และราคา



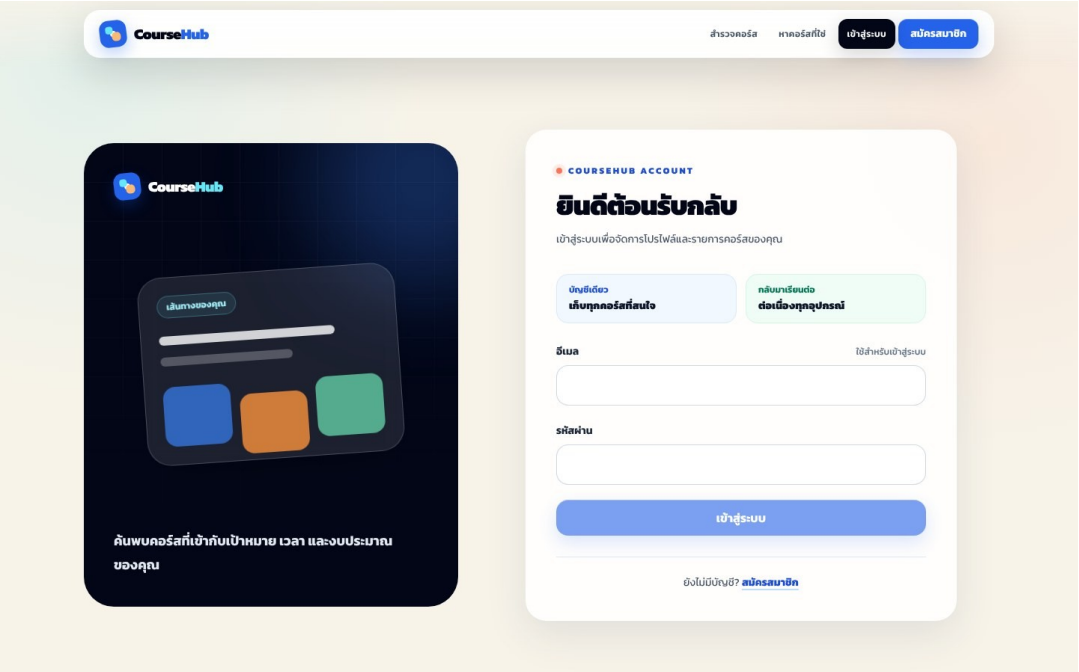
ภาพที่ 4.2 หน้าสำรวจและค้นหาคอร์ส

หน้าแบบทดสอบหาคอร์สที่ใช่ในภาพที่ 4.3 แบ่งคำถามเป็น 4 ขั้น ได้แก่ หมวดหมู่ ระดับและภาษา งบประมาณ และเวลาเรียน เมื่อส่งคำตอบแล้วระบบแสดงคอร์สไม่เกิน 3 อันดับพร้อมคะแนนรวมและเหตุผลของแต่ละด้าน

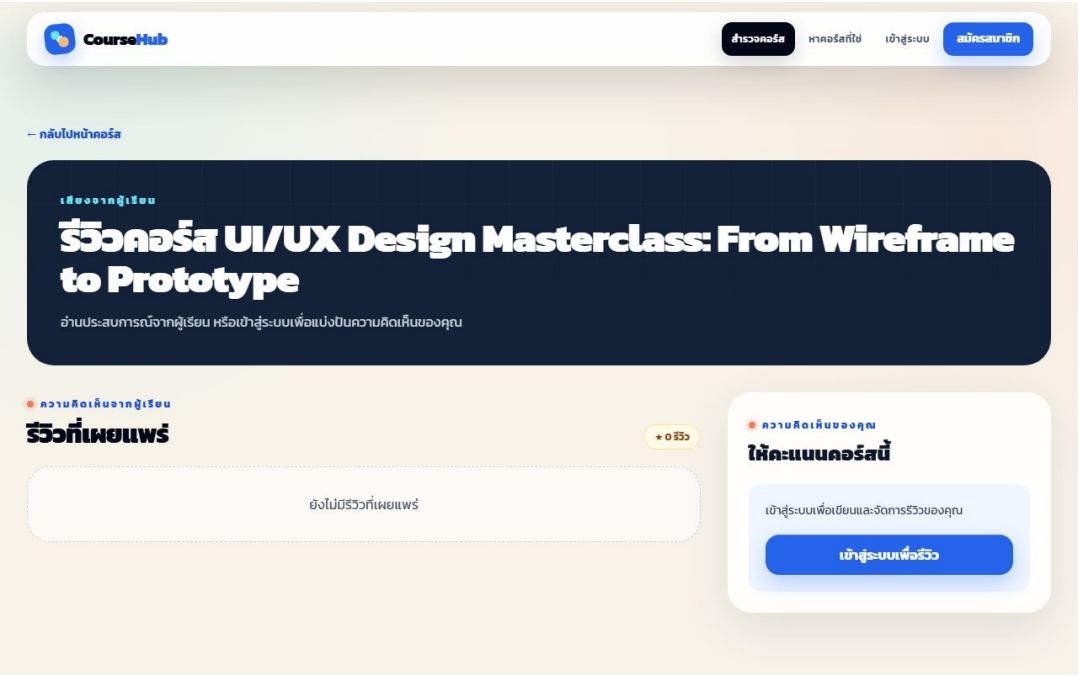


ภาพที่ 4.3 หน้าแบบทดสอบ Course Matcher

ผู้เรียนที่เข้าสู่ระบบผ่านหน้าในภาพที่ 4.4 สามารถบันทึกคอร์สไว้ดูภายหลังที่หน้า /bookmarks ซึ่งรายการยังคงอยู่หลังรีเฟรชหน้าเว็บ และอ่านหรือเขียนรีวิวในหน้ารีวิวของคอร์สดังภาพที่ 4.5 รีวิวใหม่และรีวิวที่แก้ไขจะกลับไปรอผู้ดูแลตรวจเสมอ และคะแนนเฉลี่ยนับเฉพาะรีวิวที่อนุมัติแล้ว



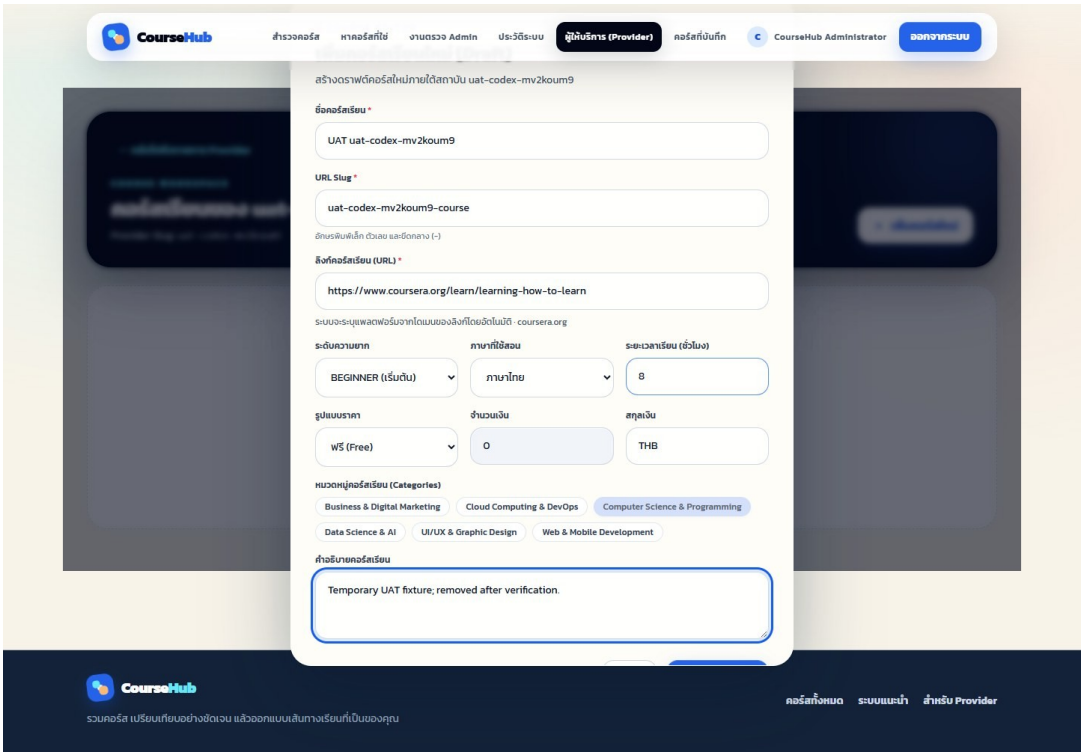
ภาพที่ 4.4 หน้าเข้าสู่ระบบ



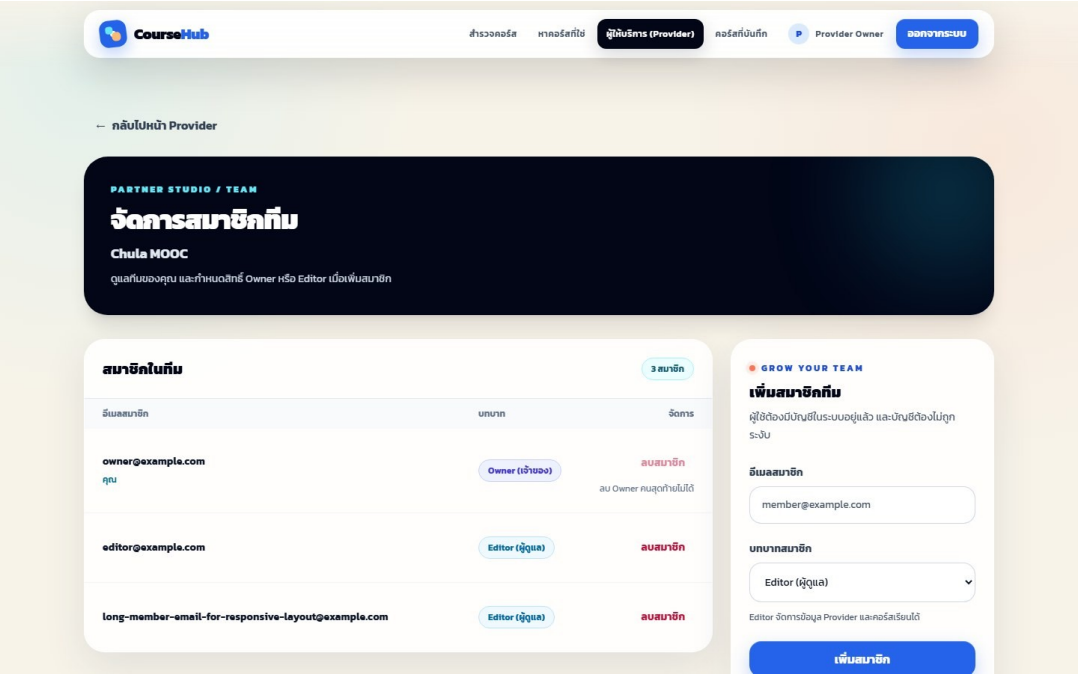
ภาพที่ 4.5 หน้ารีวิวของคอร์ส

4.1.2 ส่วนของผู้ให้บริการ

ผู้ให้บริการจัดการสถาบันที่ตนสังกัดได้ที่หน้า /providers ซึ่งแสดงสถานะการรับรองและบทบาทของผู้ใช้ ผู้ใช้บทบาท Owner หรือ Editor สร้างคอร์สแบบร่างผ่านแบบฟอร์มในภาพที่ 4.6 ระบบระบุแพลตฟอร์มจากโดเมนของ URL โดยอัตโนมัติ และให้เลือกหมวดหมู่ได้หลายหมวด ส่วนผู้ใช้บทบาท Owner จัดการสมาชิกทีมได้ที่หน้าในภาพที่ 4.7 โดยเพิ่มสมาชิกด้วยอีเมลพร้อมเลือกบทบาท และลบสมาชิกได้ตราบที่ยังมี Owner เหลืออยู่



ภาพที่ 4.6 แบบฟอร์มสร้างคอร์สแบบร่าง



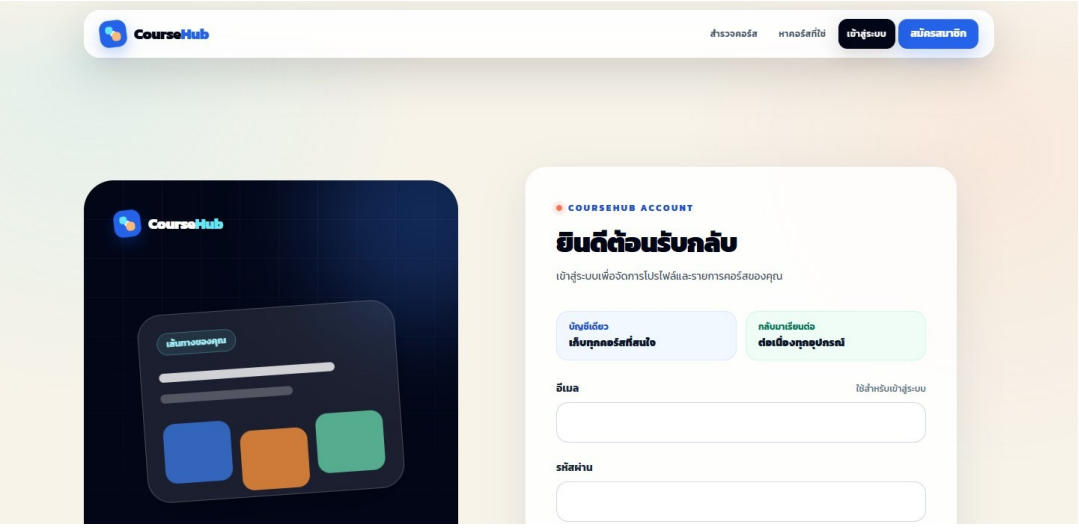
ภาพที่ 4.7 หน้าจัดการสมาชิกทีมของผู้ให้บริการ

4.1.3 ส่วนของผู้ดูแลระบบ

ผู้ดูแลระบบเข้าหน้า /admin เพื่อดูคอร์สที่รอตรวจ รับรองหรือระงับผู้ให้บริการ และตรวจคิวรีวิว การขอแก้ไข การระงับ การเก็บถาวร และการปฏิเสธรีวิวต้องระบุเหตุผลทุกครั้ง หากมีผู้อื่นแก้ข้อมูลระหว่างที่หน้าเปิดอยู่ ระบบตอบ 409 ให้โหลดข้อมูลใหม่ นอกจากนี้ผู้ดูแลเปิดหน้า /admin/audit-logs เพื่อดูประวัติการใช้งานจากฐานข้อมูล และกรองตามประเภท กิจกรรม ผู้กระทำ รายการ และช่วงเวลา พร้อมการแบ่งหน้าที่ฝั่งเซิร์ฟเวอร์

4.1.4 การรองรับช่วงเริ่มทำงานของ Backend

เนื่องจาก Backend บน Render ระดับใช้ฟรีหยุดพักเมื่อไม่มีผู้ใช้งาน หน้าเว็บจึงแสดงโครงหน้าได้ทันทีและแสดงข้อความกำลังเตรียมระบบระหว่างรอ หากการเชื่อมต่อล้มเหลว ระบบแสดงข้อความแจ้งพร้อมปุ่มลองใหม่ดังภาพที่ 4.8 เมื่อกดลองใหม่และ Backend พร้อมแล้ว หน้าเว็บกลับมาทำงานได้ตามปกติ



ภาพที่ 4.8 หน้าแจ้งการเชื่อมต่อไม่สำเร็จพร้อมปุ่มลองใหม่

4.2 ผลการทดสอบระบบ

4.2.1 ผลการทดสอบอัตโนมัติ

ผลการทดสอบอัตโนมัติรอบล่าสุดของแต่ละชุดแสดงในตารางที่ 4.1 ทุกชุดผ่านทั้งหมดโดยไม่มีกรณีที่ล้มเหลวหรือถูกข้าม ชุด Backend รันด้วยคำสั่ง mvnw verify ชุด E2E รันใน CI ของ Pull Request ที่ commit b7b1c34 บนฐานข้อมูลที่สร้างใหม่ทุกรอบ

ตารางที่ 4.1 ผลการทดสอบอัตโนมัติ

ชุดทดสอบ	เครื่องมือ	จำนวนกรณี	ผลการทดสอบ
Backend	JUnit 5, Mockito, MockMvc, Testcontainers	317	ผ่าน 317 กรณี
Frontend	Vitest, React Testing Library	106	ผ่าน 106 กรณี
End-to-End	Playwright, Chromium, PostgreSQL ใน Docker	20	ผ่าน 20 กรณี
สคริปต์ฐานข้อมูล	สคริปต์ทดสอบ PowerShell 7	8	ผ่าน 8 กรณี

4.2.2 ผลการทดสอบตามเกณฑ์การยอมรับ

คณะผู้จัดทำกำหนดเกณฑ์การยอมรับของแต่ละฟีเจอร์ไว้ในเอกสาร doc/test-plan.md และเขียนกรณีทดสอบแบบ Given When Then ตัวอย่างเกณฑ์และผลแสดงในตารางที่ 4.2

ตารางที่ 4.2 ตัวอย่างเกณฑ์การยอมรับและผลการทดสอบ

ฟีเจอร์	เกณฑ์การผ่าน	ผล
เข้าสู่ระบบ	เข้าสู่ระบบด้วยอีเมลและรหัสผ่านที่ถูกต้องได้ หากรหัสผ่านผิดต้องได้รับ 401	ผ่าน
CSRF	คำขอ POST, PUT และ DELETE ที่ไม่มี CSRF token ต้องถูกปฏิเสธ	ผ่าน
Catalog	แสดงเฉพาะคอร์สสถานะ PUBLISHED จากผู้ให้บริการที่ ACTIVE	ผ่าน
Bookmark	บันทึกซ้ำต้องไม่สร้างข้อมูลซ้ำ และผู้ใช้เห็นเฉพาะรายการของตนเอง	ผ่าน
Course CRUD	ลบได้เฉพาะคอร์สแบบร่างที่ไม่เคยเผยแพร่และไม่มีรีวิว กรณีอื่นตอบ 409	ผ่าน
Course Moderation	ผู้ดูแลอนุมัติ ขอแก้ไข ระงับ และเก็บถาวรได้ตามลำดับสถานะที่อนุญาต	ผ่าน
Review Moderation	คะแนนเฉลี่ยนับเฉพาะรีวิวที่อนุมัติแล้ว	ผ่าน
Matcher	กรองงบประมาณ ภาษา และระดับก่อนนำคอร์สที่ผ่านมาจัดอันดับ	ผ่าน
ข้อมูลลับ	API response ต้องไม่มีรหัสผ่าน ค่า hash หรือ CSRF token	ผ่าน

4.2.3 ผลการทดสอบการแนะนำคอร์สบนระบบจริง

คณะผู้จัดทำทดสอบแบบทดสอบ Matcher บนระบบที่ให้บริการจริง โดยตอบหมวดหมู่ Programming ระดับเริ่มต้น ภาษาไทย งบประมาณ 3,000 บาท และเวลาเรียน 5 ชั่วโมงต่อสัปดาห์ ระบบแนะนำคอร์ส Full-Stack Web Development Bootcamp ราคา 2,490 บาท ระยะเวลา 45 ชั่วโมง และมีคะแนนรีวิวเฉลี่ย 5.00 เป็นอันดับที่ 1 การคำนวณคะแนนแสดงในตารางที่ 4.3 คะแนนรวม 53.81 จาก 100 ตรงกับค่าที่หน้าเว็บแสดง

ตารางที่ 4.3 การคำนวณคะแนนของคอร์สอันดับที่ 1

ด้าน	การคำนวณ	คะแนน
Budget	100 × [1 − 2,490 ÷ 3,000]	17.00
Effort	เวลาที่มี 5 × 4 = 20 ชั่วโมง คิดเป็น 100 × 20 ÷ 45	44.44
Review Quality	5.00 ÷ 5 × 100	100.00
คะแนนรวม	ค่าเฉลี่ยของ 3 ด้าน	53.81

4.2.4 ผลการทดสอบ smoke บนระบบจริง

สคริปต์ test/smoke/production-readonly.mjs ส่งเฉพาะคำขอ GET ที่ไม่เปลี่ยนข้อมูลไปยังระบบจริง ครอบคลุมหน้าแรก ไฟล์ภาพ การเปิดหน้า /match โดยตรงและรีเฟรช liveness ของ Render และของ Vercel Swagger UI รายการคอร์ส หมวดหมู่ แพลตฟอร์ม เอกสาร OpenAPI การปฏิเสธผู้เยี่ยมชม และ cookie ของ CSRF ผลการทดสอบวันที่ 9 ตุลาคม 2569 ผ่าน 14 จาก 14 คำขอ และการเปิดหน้าเว็บด้วย Chromium ผ่าน 3 จาก 3 กรณี คำขอแรกที่ปลุก Render ใช้เวลาประมาณ 107 วินาที ซึ่งยืนยันว่าต้องมีหน้าโหลดและปุ่มลองใหม่

4.2.5 ผลการทดสอบการยอมรับของผู้ใช้

คณะผู้จัดทำทดสอบการยอมรับบนระบบจริงด้วยบัญชีทดสอบที่สร้างขึ้นเฉพาะ และเก็บกวาดข้อมูลทดสอบหลังจบงาน โดยไม่บันทึกรหัสผ่านหรือค่า cookie ลงใน repository รอบวันที่ 10 ตุลาคม 2569 ผ่าน 28 จุดตรวจ และมี 1 จุดที่ยังไม่ได้สังเกต คือหน้าเว็บระหว่างที่ Render เริ่มทำงานจริง ผลสรุปแสดงในตารางที่ 4.4

ตารางที่ 4.4 สรุปผลการทดสอบการยอมรับบนระบบจริง

เส้นทางการใช้งาน	สิ่งที่ตรวจ	ผล
สมัครและเข้าสู่ระบบ	สมัคร เข้าสู่ระบบ รีเฟรช ออกจากระบบ และเข้าสู่ระบบใหม่ session cookie มี Secure, HttpOnly และ SameSite=Lax	ผ่าน
สิทธิ์การเข้าถึง	ผู้เยี่ยมชมถูกส่งไปหน้าเข้าสู่ระบบ ผู้เรียนเรียก API ของผู้ดูแลได้ 403	ผ่าน
โปรไฟล์	อ่าน แก้ไข โหลดซ้ำ และคืนค่าเดิม	ผ่าน
ค้นหาและแนะนำคอร์ส	ค้นหา กรองภาษา และทำแบบทดสอบทั้งกรณีพบและไม่พบคอร์ส	ผ่าน
บันทึกคอร์ส	บันทึก รีเฟรชแล้วรายการยังอยู่ ลบ และบัญชีอื่นมองไม่เห็น	ผ่าน
รีวิว	สร้างและแก้ไขรีวิว รีวิวใหม่เป็น PENDING และไม่แสดงสาธารณะ	ผ่าน
ผู้ให้บริการและคอร์ส	สร้าง แก้ไข ลบผู้ให้บริการและคอร์สแบบร่าง ส่งตรวจก่อนได้รับการรับรองได้ 409	ผ่าน
สมาชิกทีม	Owner เพิ่มและลบ Editor ผ่านหน้าเว็บ และ Editor จัดการสมาชิกไม่ได้	ผ่าน
งานของผู้ดูแล	รับรองผู้ให้บริการ เผยแพร่คอร์ส อนุมัติและปฏิเสธรีวิว และกรองประวัติการใช้งาน	ผ่าน
ช่วงเริ่มทำงานของ Backend	หน้าเว็บระหว่างที่ Render เริ่มทำงานจริงและการไม่ส่งคำขอซ้ำ	ยังไม่ได้สังเกต

4.2.6 ผลการทดสอบการสำรองและกู้คืนข้อมูล

ชุดทดสอบ BackupRestoreRehearsalIntegrationTests สร้างฐานข้อมูล PostgreSQL 2 ชุด แล้วสำรองด้วย pg_dump และกู้คืนลงฐานข้อมูลว่างด้วย pg_restore ผ่านทั้ง 4 กรณี ได้แก่ จำนวนแถวทุกตารางตรงกับต้นทาง คอร์สยังมีราคาและหมวดหมู่ครบ Flyway ไม่ migrate ซ้ำ และ sequence ทำงานต่อจากรหัสเดิม นอกจากนี้การสำรองและกู้คืนจากฐานข้อมูล Neon จริงตรวจตรงกันครบ 13 จาก 13 ตาราง ซึ่งรวมตาราง flyway_schema_history

4.3 ผลของกระบวนการ CI/CD

GitHub Actions รอบที่ 94 ของ commit f4dd613 บน branch develop ผ่านงานทดสอบทั้ง 4 งาน และ deploy ขึ้นทั้ง Render และ Vercel สำเร็จ โดยงาน deploy ของ Vercel ล้มเหลวที่ขั้นตอนดึงค่าตั้งค่าในครั้งแรกและสำเร็จเมื่อรันซ้ำ จากนั้น alias ไปยังโดเมน group36-coursehub.vercel.app ส่วน Render ยืนยันว่ารับ deploy จาก deploy hook เท่านั้น เพราะปิดการ deploy อัตโนมัติของ Render ไว้ ทำให้ระบบขึ้นใช้งานเฉพาะ commit ที่ผ่านการทดสอบแล้ว

4.4 ผลการทำงานร่วมกันผ่าน Git

สมาชิกทุกคนรวมงานเข้า develop ผ่าน Pull Request ที่ต้องมีผู้ตรวจทานอย่างน้อย 1 คนและต้องผ่าน CI จากประวัติของ repository ณ วันที่ 10 ตุลาคม 2569 มี Pull Request ที่รวมเข้าแล้ว 39 รายการ จำนวน commit ของสมาชิกแต่ละคนแสดงในตารางที่ 4.5 โดยไม่นับ merge commit และรวมทุกบัญชี Git ที่สมาชิกคนเดียวกันใช้

ตารางที่ 4.5 จำนวน commit ของสมาชิกแต่ละคน

สมาชิก	branch	จำนวน commit
นายศุภวัฒน์ ข่ายทอง	supawat_6733800622_01	43
นายสรวิชญ์ วันเสน	sorawit_6733800648_01	73
นายเกียรติศักดิ์ นันทรัตน์	keattisak_6733800729_01	53
นายภาคิน เมฆสุวรรณ	phakin_6733800541	56

บทที่ 5
สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

คณะผู้จัดทำพัฒนาระบบ CourseHub หรือคอร์สดีบอกต่อได้ตามวัตถุประสงค์ ระบบให้บริการจริงบนคลาวด์และรองรับกรณีการใช้งาน 17 จาก 19 กรณี สรุปผลตามวัตถุประสงค์ได้ดังนี้

5.1.1 ระบบรวบรวมคอร์สจากหลายแพลตฟอร์ม ให้ผู้เรียนค้นหา กรอง เรียงลำดับ แบ่งหน้า บันทึกคอร์ส และเขียนรีวิวได้ในที่เดียว โดยแสดงเฉพาะคอร์สและรีวิวที่ผ่านการตรวจแล้ว

5.1.2 ระบบแนะนำคอร์สกรองเงื่อนไขก่อนแล้วจัดอันดับด้วย Strategy 3 ตัว แนะนำได้สูงสุด 3 คอร์ส พร้อมเหตุผล และผลบนระบบจริงตรวจสอบย้อนกลับด้วยสูตรเดียวกันได้

5.1.3 ผู้ดูแลตรวจอนุมัติคอร์ส รับรองผู้ให้บริการ ตรวจรีวิว และดูประวัติการใช้งานได้ ทุกการเปลี่ยนสถานะถูกบันทึกลง Audit Log พร้อมผู้กระทำและเหตุผล

5.1.4 ระบบใช้สถาปัตยกรรมแบบหลายชั้นที่ไม่เรียกข้ามชั้น มีหลักฐานของหลักการ SOLID ครบ 5 ข้อ และใช้รูปแบบ Strategy State และ Observer จากกลุ่ม Behavioral กับปัญหาจริงของระบบ

5.1.5 ระบบผ่านการทดสอบอัตโนมัติ Backend 317 กรณี Frontend 106 กรณี E2E 20 กรณี และสคริปต์ฐานข้อมูล 8 กรณี ผ่านการทดสอบ smoke และการทดสอบการยอมรับบนระบบจริง และ deploy อัตโนมัติผ่าน GitHub Actions

5.2 อภิปรายผล

การแยกชั้นและการใช้ DTO ทำให้ต้องเขียนโค้ดแปลงข้อมูลเพิ่มขึ้น แต่ช่วยให้ทดสอบ Service ได้โดยไม่ต้องเปิดเซิร์ฟเวอร์ และป้องกันข้อมูลภายในหลุดออกทาง API ได้จริง ซึ่งสอดคล้องกับแนวคิดของ Fowler (2002) ที่ให้ Service Layer เป็นขอบเขตของตรรกะทางธุรกิจ ส่วนการเลือกรูปแบบการออกแบบ คณะผู้จัดทำเลือกเฉพาะรูปแบบที่ตอบปัญหาของระบบ State ทำให้กติกาของ 6 สถานะอยู่ในคลาสของสถานะนั้นและตรวจด้วยการทดสอบระดับหน่วยได้ Strategy ทำให้เพิ่มสูตรคะแนนได้โดยไม่แก้ผู้เรียก ซึ่งตรงกับหลัก Open/Closed และ Observer แยกงานสถิติออกจากธุรกรรมหลัก โดยแลกกับจำนวนคลาสที่เพิ่มขึ้นและความจำเป็นที่สมาชิกทุกคนต้องเข้าใจลำดับการทำงานของเหตุการณ์

ระบบแนะนำคอร์สตั้งสมมติฐานว่าผู้เรียนต้องการเรียนจบภายใน 4 สัปดาห์ และให้คะแนนกลาง 50 เมื่อไม่ทราบระยะเวลาหรือไม่มีรีวิว สมมติฐานนี้ทำให้ผลอธิบายได้และตรวจสอบได้ แต่ไม่ได้รับประกันคุณภาพของคอร์สหรือผลการเรียน ระบบจึงแสดงเหตุผลของทุกคะแนนให้ผู้เรียนพิจารณาเอง และไม่ผ่อนเงื่อนไขเมื่อไม่พบคอร์ส

การใช้บริการคลาวด์ระดับใช้ฟรีทำให้ค่าใช้จ่ายเป็นศูนย์ แต่ต้องรับมือกับช่วงเริ่มทำงานของ Render ที่ใช้เวลาถึงประมาณ 107 วินาทีในคำขอแรก การออกแบบหน้าโหลดและปุ่มลองใหม่ช่วยให้ผู้ใช้ไม่เข้าใจผิดว่าระบบขัดข้อง ส่วนการทดสอบหลายระดับร่วมกับ CI ที่บังคับให้ทดสอบผ่านก่อน deploy ทำให้ทีมพบข้อผิดพลาดก่อนขึ้นระบบจริง

5.3 ปัญหาและอุปสรรค

5.3.1 แผนเดิมของทีมเลือกพัฒนา Backend ด้วยภาษา Rust ซึ่งขัดกับข้อกำหนดของรายวิชา ทีมจึงต้องปรับแผนเป็น Spring Boot และ Spring Data JPA ตั้งแต่ช่วงต้นโครงงาน และบันทึกเหตุผลไว้ในเอกสารการตัดสินใจเชิงสถาปัตยกรรม

5.3.2 Render และ Neon ระดับใช้ฟรีหยุดพักเมื่อไม่มีผู้ใช้งาน ทำให้คำขอแรกตอบช้า ทีมแก้โดยแสดงหน้าโหลด ตรวจ liveness และให้ผู้ใช้กดลองใหม่ได้

5.3.3 Frontend และ Backend อยู่คนละโดเมน ทำให้ cookie ข้ามโดเมนและ CORS เป็นปัญหา ทีมแก้โดยใช้ Vercel rewrite ให้เบราว์เซอร์เรียก API ผ่านโดเมนเดียวกัน

5.3.4 เครื่องของสมาชิกบางคนไม่มี Docker หรือ Java จึงรันการทดสอบที่ต้องใช้ PostgreSQL จริงบนเครื่องตนเองไม่ได้ ทีมจึงใช้ผลจาก CI เป็นหลักฐานของการทดสอบชุดดังกล่าว

5.3.5 การทดสอบบางชุดที่ใช้ฐานข้อมูล H2 ร่วมกันได้ผลขึ้นกับลำดับการรัน เพราะนับข้อมูลที่ชุดทดสอบอื่นสร้างไว้ ทีมจึงต้องระบุสาเหตุและแยกปัญหานี้ออกจากงานที่กำลังตรวจ

5.3.6 ระบบยังไม่มี API สำหรับลบบัญชีและรีวิว การเก็บกวาดข้อมูลทดสอบบนระบบจริงจึงต้องระงับบัญชีและประสานผู้ดูแลฐานข้อมูล

5.4 ข้อเสนอแนะและแนวทางพัฒนาต่อ

5.4.1 พัฒนากรณีการใช้งาน UC03 การดูเส้นทางการเรียนรู้สายอาชีพ และ UC18 การจัดการหมวดหมู่และทักษะโดยผู้ดูแลระบบ

5.4.2 เพิ่ม endpoint สำหรับลิงก์ออกไปยังเว็บไซต์ต้นทางที่นับจำนวนคลิก เพื่อวิเคราะห์ความสนใจของผู้เรียน

5.4.3 เพิ่มการเข้าสู่ระบบด้วย Google OAuth การอัปโหลดรูปปกคอร์สและรูปประจำตัว และหน้าแดชบอร์ดสถิติสำหรับผู้ให้บริการและผู้ดูแล

5.4.4 ขยายระบบแนะนำคอร์สให้รองรับค่าใช้จ่ายรวมของคอร์สแบบสมาชิกรายเดือนและการแปลงสกุลเงิน

5.4.5 เพิ่มการทดสอบระดับหน่วยด้วย Mockito ในโดเมนอื่นนอกเหนือจากคอร์ส และเพิ่ม Frontend ลงใน docker-compose.yml ให้เริ่มระบบทั้งหมดได้ด้วยคำสั่งเดียว

5.4.6 ทดสอบหน้าเว็บระหว่างที่ Render เริ่มทำงานจริงให้ครบ และย้ายบัญชีทดสอบไปเก็บในระบบจัดการรหัสผ่านของทีมก่อนการทดสอบรอบถัดไป

บรรณานุกรม

กลุ่ม 36. (2569). **GROUP36_CourseRecommend: ระบบแนะนำคอร์สเรียนคอร์สดีบอกต่อ**. ค้นเมื่อ 10 ตุลาคม 2569, จาก https://github.com/poohlikung/GROUP36_CourseRecommend

Cohn, M. (2009). **Succeeding with agile: Software development using Scrum**. Upper Saddle River, NJ: Addison-Wesley.

Docker. (n.d.). **Docker Compose overview**. Retrieved October 10, 2026, from https://docs.docker.com/compose/

Fielding, R. T. (2000). **Architectural styles and the design of network-based software architectures**. Doctoral dissertation, University of California, Irvine.

Fowler, M. (2002). **Patterns of enterprise application architecture**. Boston, MA: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). **Design patterns: Elements of reusable object-oriented software**. Reading, MA: Addison-Wesley.

GitHub. (n.d.). **GitHub Actions documentation**. Retrieved October 10, 2026, from https://docs.github.com/en/actions

Martin, R. C. (2017). **Clean architecture: A craftsman's guide to software structure and design**. Boston, MA: Prentice Hall.

Meta Platforms. (n.d.). **React**. Retrieved October 10, 2026, from https://react.dev/

Microsoft. (n.d.). **Playwright documentation**. Retrieved October 10, 2026, from https://playwright.dev/docs/intro

Neon. (n.d.). **Neon documentation**. Retrieved October 10, 2026, from https://neon.com/docs/introduction

OWASP Foundation. (n.d.). **Cross-site request forgery prevention cheat sheet**. Retrieved October 10, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Redgate Software. (n.d.). **Flyway documentation**. Retrieved October 10, 2026, from https://documentation.red-gate.com/flyway

Render. (n.d.). **Deploy for free**. Retrieved October 10, 2026, from https://render.com/docs/free

Spring. (n.d.-a). **Spring Framework documentation: The IoC container**. Retrieved October 10, 2026, from https://docs.spring.io/spring-framework/reference/core/beans.html

_____. (n.d.-b). **Spring Security reference**. Retrieved October 10, 2026, from https://docs.spring.io/spring-security/reference/index.html

Testcontainers. (n.d.). **Testcontainers for Java**. Retrieved October 10, 2026, from https://java.testcontainers.org/

The JUnit Team. (n.d.). **JUnit 5 user guide**. Retrieved October 10, 2026, from https://junit.org/junit5/docs/current/user-guide/

The PostgreSQL Global Development Group. (n.d.). **PostgreSQL documentation**. Retrieved October 10, 2026, from https://www.postgresql.org/docs/

Vercel. (n.d.). **Rewrites**. Retrieved October 10, 2026, from https://vercel.com/docs/rewrites

ภาคผนวก

ภาคผนวก ก
การติดตั้งและเรียกใช้งานระบบ

ก.1 ซอฟต์แวร์ที่ต้องติดตั้ง

เครื่องที่ใช้รันระบบต้องติดตั้ง Git, Docker Desktop หรือ Docker Engine พร้อม Docker Compose, Java 21 และ Node.js รุ่น 22 ขึ้นไปพร้อม npm

ก.2 การเริ่มระบบ Backend และฐานข้อมูล

1. โคลน repository แล้วเข้าไปยังโฟลเดอร์ของโครงงาน
   git clone https://github.com/poohlikung/GROUP36_CourseRecommend.git
2. คัดลอกไฟล์ .env.example เป็น .env แล้วตั้งค่า POSTGRES_PASSWORD สำหรับเครื่องของตนเอง หากรันผ่าน HTTP บนเครื่องให้ตั้ง SESSION_COOKIE_SECURE=false
3. เริ่มฐานข้อมูล PostgreSQL และ Backend ด้วยคำสั่งต่อไปนี้ Flyway จะสร้างตารางให้อัตโนมัติเมื่อ Backend เริ่มทำงาน
   docker compose up --build
4. ตรวจสถานะ Backend ที่ http://localhost:8080/api/v1/system/liveness และเปิดเอกสาร API ที่ http://localhost:8080/swagger-ui.html

ก.3 การเริ่มระบบ Frontend

Frontend ใช้ Vite proxy ส่งคำขอ /api ไปยัง Backend บนเครื่อง เริ่มทำงานด้วยคำสั่งต่อไปนี้ แล้วเปิดเบราว์เซอร์ที่ http://localhost:5173

```
cd code/frontend
npm install
npm run dev
```

ก.4 การรันชุดทดสอบ

1. ทดสอบ Backend จากโฟลเดอร์ code/backend ด้วยคำสั่ง ./mvnw verify บน Linux หรือ macOS และ .\mvnw.cmd verify บน Windows ชุดทดสอบที่ใช้ PostgreSQL ต้องเปิด Docker ไว้
2. ทดสอบ Frontend จากโฟลเดอร์ code/frontend ด้วยคำสั่ง npm run typecheck, npm test และ npm run build
3. ทดสอบ E2E ด้วยคำสั่ง npm run test:e2e ในโฟลเดอร์ code/frontend ซึ่งสร้างฐานข้อมูลทดสอบแยกใน Docker และลบทิ้งเมื่อจบการทดสอบ

ก.5 ที่อยู่ของระบบที่ให้บริการจริง

1. หน้าเว็บ https://group36-coursehub.vercel.app
2. เอกสาร API https://coursehub-backend-ahz2.onrender.com/swagger-ui.html
3. repository https://github.com/poohlikung/GROUP36_CourseRecommend